

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS

Escola Politécnica e de Artes

Curso Superior de Tecnologia em Desenvolvimento de Sistemas

Material de Aula — Design de Software
DESIGN PATTERNS EM JAVA

Catálogo completo dos 23 Padrões GoF com exemplos contextualizados

Disciplina	Design de Software
Curso	CST em Desenvolvimento de Sistemas
Conteúdo	Padrões de Projeto GoF — Criacionais, Estruturais e Comportamentais
Linguagem	Java (JDK 17+)
Total de Padrões	23 padrões com 3 exemplos contextualizados cada

Sumário dos Padrões GoF

Categoria	Padrão	Objetivo Principal
CRIACIONAIS	Singleton	Instância única global
	Factory Method	Criação delegada à subclasse
	Abstract Factory	Famílias de objetos relacionados
	Builder	Construção passo a passo
	Prototype	Clonagem de objetos
ESTRUTURAIS	Adapter	Compatibilidade de interfaces
	Bridge	Separação abstração/implementação
	Composite	Hierarquias parte-todo
	Decorator	Responsabilidades dinâmicas
	Facade	Interface simplificada para subsistemas
	Flyweight	Compartilhamento eficiente de objetos
	Proxy	Representante/controlador de acesso
COMPORTAMENTAIS	Chain of Responsibility	Cadeia de handlers
	Command	Ações encapsuladas como objetos
	Iterator	Acesso sequencial sem expor estrutura
	Mediator	Comunicação centralizada
	Memento	Captura e restauração de estado
	Observer	Notificação automática de dependentes
	State	Comportamento baseado em estado
	Strategy	Família de algoritmos intercambiáveis
	Template Method	Esqueleto de algoritmo fixo
	Visitor	Operações sobre estrutura sem modificá-la
	Interpreter	Interpretação de linguagem/gramática

Introdução aos Design Patterns

O que são Design Patterns?

Design Patterns (Padrões de Projeto) são soluções reutilizáveis para problemas recorrentes no desenvolvimento de software orientado a objetos. Não são algoritmos ou código pronto — são modelos conceituais que descrevem a estrutura e o relacionamento entre classes e objetos para resolver um determinado tipo de problema.

O conceito foi popularizado pelo livro "Design Patterns: Elements of Reusable Object-Oriented Software" (1994), de Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides — o famoso "Gang of Four" (GoF). O catálogo GoF apresenta 23 padrões divididos em três categorias.

Benefícios do Uso de Padrões

- Vocabulário comum: facilita a comunicação entre desenvolvedores da equipe.
- Reutilização: soluções testadas e comprovadas para problemas conhecidos.
- Manutenibilidade: código mais desacoplado e de fácil manutenção.
- Extensibilidade: facilita a adição de novas funcionalidades com baixo impacto.
- Qualidade: reduz a probabilidade de introduzir bugs ao resolver problemas clássicos.

As Três Categorias GoF

Categoria	Qtde	Foco
Criacionais	5 padrões	Como objetos são criados — controla instanciação e composição de objetos.
Estruturais	7 padrões	Como objetos são compostos — define relações entre classes e objetos.
Comportamentais	11 padrões	Como objetos interagem — define responsabilidades e algoritmos.

Como Usar Este Material

Cada padrão é apresentado com: (1) Intenção — o problema que resolve; (2) Três exemplos contextualizados em Java com código completo e comentado. Os exemplos são projetados para simular cenários reais de sistemas empresariais, acadêmicos e de software comercial.

Padrões Criacionais (Creational Patterns)

1.1 Singleton

💡 *Intenção: Garante que uma classe tenha apenas uma instância e fornece um ponto global de acesso a ela.*

Exemplo 1: Gerenciador de Conexão com Banco de Dados

Em um sistema de gestão acadêmica da PUC Goiás, a conexão com o banco de dados deve ser única para evitar desperdício de recursos. O Singleton garante que apenas um pool de conexões seja criado.

```
public class DatabaseConnection {
    private static DatabaseConnection instance;
    private Connection connection;

    private DatabaseConnection() {
        try {
            this.connection = DriverManager.getConnection(
                "jdbc:postgresql://localhost/pucgoias", "user", "pass");
        } catch (SQLException e) { e.printStackTrace(); }
    }

    public static synchronized DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }

    public Connection getConnection() { return connection; }
}

// Uso:
Connection conn = DatabaseConnection.getInstance().getConnection();
```

Exemplo 2: Logger de Auditoria do Sistema

Um sistema de e-commerce precisa registrar todas as transações em um único arquivo de log. O Singleton thread-safe (double-checked locking) é utilizado para garantir unicidade mesmo em ambientes concorrentes.

```
public class AuditLogger {
    private static volatile AuditLogger instance;
    private final List<String> logs = new ArrayList<>();

    private AuditLogger() {}

    public static AuditLogger getInstance() {
        if (instance == null) {
            synchronized (AuditLogger.class) {
                if (instance == null) {
                    instance = new AuditLogger();
                }
            }
        }
        return instance;
    }

    public synchronized void log(String msg) {
        logs.add(LocalDate.now() + " - " + msg);
    }
}
```

```
public List<String> getLogs() { return Collections.unmodifiableList(logs); }
}
```

Exemplo 3: Gerenciador de Configurações da Aplicação

Um sistema de gestão hospitalar carrega configurações de um arquivo properties uma única vez na inicialização. O Singleton assegura que todas as classes usem as mesmas configurações globais.

```
public class AppConfig {
    private static final AppConfig INSTANCE = new AppConfig();
    private final Properties props = new Properties();

    private AppConfig() {
        try (InputStream is = getClass()
            .getResourceAsStream("/config.properties")) {
            props.load(is);
        } catch (IOException e) { e.printStackTrace(); }
    }

    public static AppConfig getInstance() { return INSTANCE; }

    public String get(String key) { return props.getProperty(key); }
    public int getInt(String key) {
        return Integer.parseInt(props.getProperty(key));
    }
}

// Uso:
String url = AppConfig.getInstance().get("api.url");
```

1.2 Factory Method

💡 Intenção: Define uma interface para criar um objeto, mas deixa as subclasses decidirem qual classe instanciar. Permite que uma classe adie a instanciação para suas subclasses.

Exemplo 1: Gerador de Relatórios (PDF, Excel, CSV)

Um sistema de RH precisa gerar relatórios de funcionários em diferentes formatos. O Factory Method permite criar o gerador correto sem que o código cliente conheça as classes concretas.

```
public interface Relatorio {
    void gerar(List<Funcionario> dados);
}

public class RelatorioPDF implements Relatorio {
    public void gerar(List<Funcionario> dados) {
        System.out.println("Gerando PDF com " + dados.size() + " registros");
    }
}

public class RelatorioExcel implements Relatorio {
    public void gerar(List<Funcionario> dados) {
        System.out.println("Gerando Excel com " + dados.size() + " registros");
    }
}

public abstract class RelatorioFactory {
    public abstract Relatorio criarRelatorio();

    public void exportar(List<Funcionario> dados) {
```

```
        Relatorio r = criarRelatorio();
        r.gerar(dados);
    }
}

public class PDFFactory extends RelatorioFactory {
    public Relatorio criarRelatorio() { return new RelatorioPDF(); }
}

// Uso:
RelatorioFactory factory = new PDFFactory();
factory.exportar(listaFuncionarios);
```

Exemplo 2: Sistema de Notificações (E-mail, SMS, Push)

Um app de delivery precisa enviar notificações por diferentes canais dependendo da preferência do cliente. O Factory Method encapsula a criação do notificador adequado.

```
public interface Notificador {
    void enviar(String destinatario, String mensagem);
}

public class NotificadorEmail implements Notificador {
    public void enviar(String dest, String msg) {
        System.out.println("Email para " + dest + ": " + msg);
    }
}

public class NotificadorSMS implements Notificador {
    public void enviar(String dest, String msg) {
        System.out.println("SMS para " + dest + ": " + msg);
    }
}

public abstract class CanalNotificacao {
    protected abstract Notificador criarNotificador();

    public void notificar(String dest, String msg) {
        criarNotificador().enviar(dest, msg);
    }
}

public class CanalEmail extends CanalNotificacao {
    protected Notificador criarNotificador() { return new NotificadorEmail(); }
}

// Uso:
CanalNotificacao canal = new CanalEmail();
canal.notificar("user@puc.br", "Pedido confirmado!");
```

Exemplo 3: Parser de Documentos (JSON, XML, YAML)

Um sistema de integração recebe dados em diferentes formatos. O Factory Method cria o parser adequado conforme o tipo de arquivo recebido, sem expor a lógica de parsing ao cliente.

```
public interface DocumentParser {
    Map<String, Object> parse(String conteudo);
}

public class JsonParser implements DocumentParser {
    public Map<String, Object> parse(String c) {
        System.out.println("Parseando JSON...");
        return new HashMap<>(); // implementação real usaria Gson/Jackson
    }
}
```

```

public class XmlParser implements DocumentParser {
    public Map<String, Object> parse(String c) {
        System.out.println("Parseando XML...");
        return new HashMap<>();
    }
}

public abstract class ParserFactory {
    public abstract DocumentParser criarParser();

    public static ParserFactory para(String tipo) {
        return switch (tipo.toLowerCase()) {
            case "json" -> new JsonParserFactory();
            case "xml" -> new XmlParserFactory();
            default -> throw new IllegalArgumentException("Tipo desconhecido: " +
tipo);
        };
    }
}

class JsonParserFactory extends ParserFactory {
    public DocumentParser criarParser() { return new JsonParser(); }
}
class XmlParserFactory extends ParserFactory {
    public DocumentParser criarParser() { return new XmlParser(); }
}

```

1.3 Abstract Factory

💡 Intenção: Fornece uma interface para criar famílias de objetos relacionados ou dependentes sem especificar suas classes concretas.

Exemplo 1: UI Multi-plataforma (Windows / macOS)

Um sistema desktop precisa renderizar componentes de interface (botão, caixa de texto) de acordo com o sistema operacional. A Abstract Factory garante que todos os componentes pertençam à mesma família visual.

```

// Produtos abstratos
interface Botao { void renderizar(); }
interface CaixaTexto { void renderizar(); }

// Família Windows
class BotaoWindows implements Botao {
    public void renderizar() { System.out.println("[Windows Button]"); }
}
class CaixaTextoWindows implements CaixaTexto {
    public void renderizar() { System.out.println("[Windows TextBox]"); }
}

// Família macOS
class BotaoMac implements Botao {
    public void renderizar() { System.out.println("[macOS Button]"); }
}
class CaixaTextoMac implements CaixaTexto {
    public void renderizar() { System.out.println("[macOS TextBox]"); }
}

// Fábrica abstrata
interface UIFactory {
    Botao criarBotao();
}

```

```

        CaixaTexto criarCaixaTexto();
    }

    class WindowsFactory implements UIFactory {
        public Botao criarBotao() { return new BotaoWindows(); }
        public CaixaTexto criarCaixaTexto() { return new CaixaTextoWindows(); }
    }

    // Uso:
    UIFactory factory = new WindowsFactory();
    factory.criarBotao().renderizar();

```

Exemplo 2: Sistema de Pagamento (Crédito / Débito / PIX)

Uma plataforma financeira precisa criar validadores e processadores de pagamento em conjunto. A Abstract Factory garante que o validador e processador sejam sempre compatíveis entre si.

```

interface ValidadorPagamento { boolean validar(String dados); }
interface ProcessadorPagamento { String processar(double valor); }

// Família PIX
class ValidadorPix implements ValidadorPagamento {
    public boolean validar(String chave) {
        return chave != null && !chave.isEmpty();
    }
}
class ProcessadorPix implements ProcessadorPagamento {
    public String processar(double v) {
        return "PIX processado: R$ " + v;
    }
}

// Família Cartão de Crédito
class ValidadorCartao implements ValidadorPagamento {
    public boolean validar(String n) { return n.length() == 16; }
}
class ProcessadorCartao implements ProcessadorPagamento {
    public String processar(double v) {
        return "Cartão processado: R$ " + v;
    }
}

interface PagamentoFactory {
    ValidadorPagamento criarValidador();
    ProcessadorPagamento criarProcessador();
}

class PixFactory implements PagamentoFactory {
    public ValidadorPagamento criarValidador() { return new ValidadorPix(); }
    public ProcessadorPagamento criarProcessador() { return new ProcessadorPix(); }
}

```

Exemplo 3: Geração de Dados de Teste (Mock / Stub)

Em testes automatizados, a equipe precisa criar objetos de usuário e pedido coerentes entre si (ex.: usuários premium com pedidos de alto valor). A Abstract Factory cria famílias consistentes de dados para testes.

```

interface Usuario { String getNome(); String getTipo(); }
interface Pedido { double getValor(); String getStatus(); }

class UsuarioPremium implements Usuario {
    public String getNome() { return "Ana Lima"; }
    public String getTipo() { return "PREMIUM"; }
}
class PedidoAltoValor implements Pedido {

```

```

    public double getValor()    { return 1500.0; }
    public String getStatus()  { return "APROVADO"; }
}

class UsuarioBasico implements Usuario {
    public String getNome()    { return "Pedro Silva"; }
    public String getTipo()    { return "BASICO"; }
}

class PedidoBaixoValor implements Pedido {
    public double getValor()    { return 50.0; }
    public String getStatus()  { return "PENDENTE"; }
}

interface TestDataFactory {
    Usuario criarUsuario();
    Pedido  criarPedido();
}

class PremiumTestFactory implements TestDataFactory {
    public Usuario criarUsuario() { return new UsuarioPremium(); }
    public Pedido  criarPedido()  { return new PedidoAltoValor(); }
}

```

1.4 Builder

💡 Intenção: Separa a construção de um objeto complexo de sua representação, de forma que o mesmo processo de construção possa criar diferentes representações.

Exemplo 1: Construção de Pedido de E-commerce

Um pedido de e-commerce possui muitos campos opcionais (cupom, endereço de presente, seguro, etc.). O Builder evita construtores com dezenas de parâmetros e garante que o pedido seja montado de forma legível e segura.

```

public class Pedido {
    private final String clienteId;
    private final List<Item> itens;
    private final String cupomDesconto;
    private final String enderecoEntrega;
    private final boolean seguroContratado;

    private Pedido(Builder b) {
        this.clienteId      = b.clienteId;
        this.itens           = b.itens;
        this.cupomDesconto  = b.cupomDesconto;
        this.enderecoEntrega = b.enderecoEntrega;
        this.seguroContratado = b.seguroContratado;
    }

    public static class Builder {
        private final String clienteId;
        private List<Item> itens = new ArrayList<>();
        private String cupomDesconto;
        private String enderecoEntrega;
        private boolean seguroContratado = false;

        public Builder(String clienteId) { this.clienteId = clienteId; }
        public Builder addItem(Item i)    { itens.add(i); return this; }
        public Builder cupom(String c)    { cupomDesconto = c; return this; }
        public Builder endereco(String e) { enderecoEntrega = e; return this; }
        public Builder comSeguro()        { seguroContratado = true; return this; }
    }
}

```

```

        public Pedido build() { return new Pedido(this); }
    }

// Uso:
Pedido p = new Pedido.Builder("CLI-42")
    .addItem(new Item("Notebook", 3000.0))
    .cupom("DESCONTO10")
    .endereço("Rua 1, Goiânia")
    .comSeguro()
    .build();

```

Exemplo 2: Montagem de Query SQL Dinâmica

Um sistema de relatórios permite ao usuário filtrar dados com critérios variáveis. O Builder constrói a query dinamicamente sem concatenações frágeis de strings.

```

public class QueryBuilder {
    private String tabela;
    private final List<String> colunas = new ArrayList<>();
    private final List<String> condicoes = new ArrayList<>();
    private String ordenacao;
    private int limite = -1;

    public QueryBuilder from(String t) { tabela = t; return this; }
    public QueryBuilder select(String col) { colunas.add(col); return this; }
    public QueryBuilder where(String cond) { condicoes.add(cond); return this; }
    public QueryBuilder orderBy(String col) { ordenacao = col; return this; }
    public QueryBuilder limit(int n) { limite = n; return this; }

    public String build() {
        StringBuilder sb = new StringBuilder("SELECT ");
        sb.append(colunas.isEmpty() ? "*" : String.join(", ", colunas));
        sb.append(" FROM ").append(tabela);
        if (!condicoes.isEmpty())
            sb.append(" WHERE ").append(String.join(" AND ", condicoes));
        if (ordenacao != null) sb.append(" ORDER BY ").append(ordenacao);
        if (limite > 0) sb.append(" LIMIT ").append(limite);
        return sb.toString();
    }
}

// Uso:
String sql = new QueryBuilder()
    .from("alunos")
    .select("nome").select("ra")
    .where("curso = 'ADS'")
    .orderBy("nome")
    .limit(20)
    .build();

```

Exemplo 3: Construção de E-mail Transacional

Um sistema de notificações envia e-mails com diferentes combinações de destinatários, assunto, corpo HTML/texto, anexos e cabeçalhos. O Builder organiza essa construção complexa de forma fluente.

```

public class Email {
    private final String de, para, assunto, corpoHtml, corpoTexto;
    private final List<String> cc, bcc, anexos;

    private Email(Builder b) {
        de = b.de; para = b.para; assunto = b.assunto;
        corpoHtml = b.corpoHtml; corpoTexto = b.corpoTexto;
        cc = b.cc; bcc = b.bcc; anexos = b.anexos;
    }
}

```

```

public static class Builder {
    private String de, para, assunto, corpoHtml, corpoTexto;
    private List<String> cc = new ArrayList<>(), bcc = new ArrayList<>();
    private List<String> anexos = new ArrayList<>();

    public Builder de(String d) { de = d; return this; }
    public Builder para(String p) { para = p; return this; }
    public Builder assunto(String a) { assunto = a; return this; }
    public Builder htmlBody(String h) { corpoHtml = h; return this; }
    public Builder textBody(String t) { corpoTexto = t; return this; }
    public Builder cc(String e) { cc.add(e); return this; }
    public Builder bcc(String e) { bcc.add(e); return this; }
    public Builder anexo(String caminho) { anexos.add(caminho); return this; }
    public Email build() { return new Email(this); }
}

// Uso:
Email email = new Email.Builder()
    .de("noreply@pucgoias.edu.br")
    .para("aluno@email.com")
    .assunto("Confirmação de matrícula")
    .htmlBody("<h1>Bem-vindo!</h1>")
    .cc("coordenacao@pucgoias.edu.br")
    .build();

```

1.5 Prototype

💡 Intenção: Especifica os tipos de objetos a criar usando uma instância prototípica e cria novos objetos copiando esse protótipo.

Exemplo 1: Clonagem de Templates de Proposta Comercial

Uma empresa possui modelos padrão de proposta comercial. Ao criar uma nova proposta, ela é clonada a partir do template e depois personalizada, evitando recriar toda a estrutura do zero.

```

public class PropostaComercial implements Cloneable {
    private String cliente;
    private List<ItemProposta> itens;
    private String condicoesPagamento;
    private double desconto;

    public PropostaComercial(String cliente, List<ItemProposta> itens,
        String cond, double desc) {
        this.cliente = cliente;
        this.itens = new ArrayList<>(itens);
        this.condicoesPagamento = cond;
        this.desconto = desc;
    }

    @Override
    public PropostaComercial clone() {
        try {
            PropostaComercial copia = (PropostaComercial) super.clone();
            copia.itens = new ArrayList<>(this.itens); // cópia profunda
            return copia;
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException(e);
        }
    }
}

```

```

    public void setCliente(String c) { this.cliente = c; }
}

// Uso:
PropostaComercial template = new PropostaComercial(
    "Template Padrão", itensBase, "30/60/90 dias", 5.0);

PropostaComercial novaProposta = template.clone();
novaProposta.setCliente("Empresa XYZ");

```

Exemplo 2: Cópia de Configurações de Personagem em Jogo

Em um jogo RPG, o jogador pode criar variações de personagens baseadas em um arquétipo base. O Prototype clona o personagem base com todos os seus atributos e habilidades para personalizações rápidas.

```

public class Personagem implements Cloneable {
    private String nome;
    private int forca, agilidade, inteligencia, vida;
    private List<String> habilidades;

    public Personagem(String nome, int forca, int agi, int intel, int vida) {
        this.nome = nome; this.forca = forca; this.agilidade = agi;
        this.inteligencia = intel; this.vida = vida;
        this.habilidades = new ArrayList<>();
    }

    public void addHabilidade(String h) { habilidades.add(h); }

    @Override
    public Personagem clone() {
        try {
            Personagem c = (Personagem) super.clone();
            c.habilidades = new ArrayList<>(this.habilidades);
            return c;
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException(e);
        }
    }

    public void setNome(String n) { this.nome = n; }
    public void setForca(int f) { this.forca = f; }
}

// Uso:
Personagem guerreiroBase = new Personagem("Guerreiro", 90, 60, 40, 200);
guerreiroBase.addHabilidade("Golpe Poderoso");

Personagem guerreiroJogador = guerreiroBase.clone();
guerreiroJogador.setNome("Ragnar");
guerreiroJogador.setForca(95);

```

Exemplo 3: Duplicação de Layouts de Dashboard

Uma ferramenta de BI permite duplicar dashboards existentes para criar variações com filtros diferentes. O Prototype clona o dashboard com todos os seus widgets sem reprocessar a configuração original.

```

public class Widget implements Cloneable {
    private String tipo, titulo;
    private Map<String, String> config;

    public Widget(String tipo, String titulo, Map<String, String> config) {
        this.tipo = tipo; this.titulo = titulo;
    }
}

```

```
        this.config = new HashMap<>(config);
    }

    @Override
    public Widget clone() {
        try {
            Widget c = (Widget) super.clone();
            c.config = new HashMap<>(this.config);
            return c;
        } catch (CloneNotSupportedException e) { throw new RuntimeException(e); }
    }
}

public class Dashboard implements Cloneable {
    private String nome;
    private List<Widget> widgets;

    @Override
    public Dashboard clone() {
        try {
            Dashboard c = (Dashboard) super.clone();
            c.widgets = widgets.stream()
                               .map(Widget::clone)
                               .collect(Collectors.toList());

            return c;
        } catch (CloneNotSupportedException e) { throw new RuntimeException(e); }
    }

    public void setNome(String n) { this.nome = n; }
}
```

Padrões Estruturais (Structural Patterns)

2.1 Adapter

💡 *Intenção: Converte a interface de uma classe em outra interface que os clientes esperam. O Adapter permite que classes com interfaces incompatíveis trabalhem juntas.*

Exemplo 1: Integração com API de Pagamento Legado

Um sistema novo precisa usar um gateway de pagamento antigo que possui interface incompatível. O Adapter traduz chamadas modernas para o formato esperado pelo sistema legado.

```
// Interface que o sistema moderno espera
public interface GatewayPagamento {
    boolean processar(String clienteId, double valor, String moeda);
}

// Sistema legado (não pode ser modificado)
public class GatewayLegado {
    public int efetuarTransacao(int codCliente, long valorCentavos) {
        System.out.println("Legado: cod=" + codCliente + " val=" + valorCentavos);
        return 1; // 1 = sucesso
    }
}

// Adapter
public class GatewayAdapter implements GatewayPagamento {
    private final GatewayLegado legado = new GatewayLegado();

    @Override
    public boolean processar(String clienteId, double valor, String moeda) {
        int codCliente = Integer.parseInt(clienteId.replace("CLI-", ""));
        long centavos = Math.round(valor * 100);
        int resultado = legado.efetuarTransacao(codCliente, centavos);
        return resultado == 1;
    }
}

// Uso transparente:
GatewayPagamento gw = new GatewayAdapter();
boolean ok = gw.processar("CLI-007", 199.90, "BRL");
```

Exemplo 2: Adaptador de Biblioteca de Logs

Uma aplicação usa uma interface interna de logging, mas precisa integrar com o SLF4J sem modificar o código existente. O Adapter conecta as duas interfaces.

```
// Interface interna da aplicação
public interface AppLogger {
    void info(String msg);
    void error(String msg, Throwable t);
}

// Adapter para SLF4J
public class Slf4jAdapter implements AppLogger {
    private final org.slf4j.Logger logger;

    public Slf4jAdapter(Class<?> clazz) {
        this.logger = org.slf4j.LoggerFactory.getLogger(clazz);
    }

    @Override
```

```

    public void info(String msg) { logger.info(msg); }

    @Override
    public void error(String msg, Throwable t) { logger.error(msg, t); }
}

// Uso:
AppLogger log = new Slf4jAdapter(MeuServico.class);
log.info("Serviço iniciado com sucesso");

```

Exemplo 3: Adaptador de Formato de Data

Um microserviço recebe datas no formato ISO-8601 (String), mas uma biblioteca de relatórios trabalha com objetos java.util.Date. O Adapter realiza a conversão de forma transparente.

```

// Interface esperada pela biblioteca de relatórios
public interface FonteDados {
    Date getDataInicio();
    Date getDataFim();
}

// Dado recebido do microserviço
public class FiltroISO {
    private final String inicio; // "2025-01-01T00:00:00Z"
    private final String fim;
    public FiltroISO(String inicio, String fim) {
        this.inicio = inicio; this.fim = fim;
    }
    public String getInicioISO() { return inicio; }
    public String getFimISO() { return fim; }
}

// Adapter
public class FiltroAdapter implements FonteDados {
    private final FiltroISO filtro;
    private final DateTimeFormatter fmt =
        DateTimeFormatter.ISO_INSTANT;

    public FiltroAdapter(FiltroISO f) { this.filtro = f; }

    public Date getDataInicio() {
        return Date.from(Instant.parse(filtro.getInicioISO()));
    }
    public Date getDataFim() {
        return Date.from(Instant.parse(filtro.getFimISO()));
    }
}

```

2.2 Bridge

 *Intenção: Desacopla uma abstração de sua implementação para que as duas possam variar de forma independente.*

Exemplo 1: Sistema de Envio de Mensagens Multi-canal

Uma plataforma de notificações precisa combinar diferentes tipos de mensagem (urgente, informativo) com diferentes canais (e-mail, SMS). Bridge desacopla hierarquia de mensagens da hierarquia de canais.

```

// Implementação (canal de envio)
public interface Canal {
    void enviar(String destinatario, String texto);
}

```

```

}
public class CanalEmail implements Canal {
    public void enviar(String dest, String texto) {
        System.out.println("Email -> " + dest + ": " + texto);
    }
}
public class Canalsms implements Canal {
    public void enviar(String dest, String texto) {
        System.out.println("SMS -> " + dest + ": " + texto);
    }
}

// Abstração (tipo de mensagem)
public abstract class Mensagem {
    protected Canal canal;
    public Mensagem(Canal canal) { this.canal = canal; }
    public abstract void disparar(String dest, String conteudo);
}

public class MensagemUrgente extends Mensagem {
    public MensagemUrgente(Canal canal) { super(canal); }
    public void disparar(String dest, String c) {
        canal.enviar(dest, "[URGENTE] " + c.toUpperCase());
    }
}

public class MensagemInformativa extends Mensagem {
    public MensagemInformativa(Canal canal) { super(canal); }
    public void disparar(String dest, String c) {
        canal.enviar(dest, "[INFO] " + c);
    }
}

// Uso (combinações livres):
Mensagem m1 = new MensagemUrgente(new Canalsms());
Mensagem m2 = new MensagemInformativa(new CanalEmail());
m1.disparar("+55 62 9999", "Servidor fora do ar!");

```

Exemplo 2: Renderização de Formas Geométricas (2D/3D)

Um sistema CAD precisa renderizar formas (círculo, retângulo) em diferentes contextos gráficos (OpenGL, DirectX). Bridge separa a hierarquia de formas da hierarquia de renderizadores.

```

public interface Renderizador {
    void desenharCirculo(double x, double y, double r);
    void desenharRetangulo(double x, double y, double w, double h);
}
class RenderizadorOpenGL implements Renderizador {
    public void desenharCirculo(double x, double y, double r) {
        System.out.printf("[OpenGL] Circulo em (%.1f,%.1f) r=%.1f%n", x,y,r);
    }
    public void desenharRetangulo(double x, double y, double w, double h) {
        System.out.printf("[OpenGL] Ret em (%.1f,%.1f) %.1fx%.1f%n",x,y,w,h);
    }
}

public abstract class Forma {
    protected Renderizador renderizador;
    public Forma(Renderizador r) { this.renderizador = r; }
    public abstract void desenhar();
}
class Circulo extends Forma {
    private double x, y, raio;
    public Circulo(double x, double y, double r, Renderizador rend) {
        super(rend); this.x=x; this.y=y; this.raio=r;
    }
}

```

```

    public void desenhar() { renderizador.desenharCirculo(x, y, raio); }
}

```

Exemplo 3: Tema de Interface (Claro/Escuro) com Plataformas

Um sistema web precisa aplicar temas visuais (claro, escuro) em diferentes plataformas (desktop, mobile). Bridge combina livremente temas com plataformas sem explosão combinatória de classes.

```

public interface PlataformaUI {
    void exibirComponente(String nome, String corFundo, String corTexto);
}

class DesktopUI implements PlataformaUI {
    public void exibirComponente(String n, String bg, String fg) {
        System.out.printf("[Desktop] %s bg=%s fg=%s\n", n, bg, fg);
    }
}

class MobileUI implements PlataformaUI {
    public void exibirComponente(String n, String bg, String fg) {
        System.out.printf("[Mobile] %s bg=%s fg=%s\n", n, bg, fg);
    }
}

public abstract class Tema {
    protected PlataformaUI plataforma;
    public Tema(PlataformaUI p) { this.plataforma = p; }
    public abstract void aplicar(String componente);
}

class TemaClaro extends Tema {
    public TemaClaro(PlataformaUI p) { super(p); }
    public void aplicar(String c) {
        plataforma.exibirComponente(c, "#FFFFFF", "#000000");
    }
}

class TemaEscuro extends Tema {
    public TemaEscuro(PlataformaUI p) { super(p); }
    public void aplicar(String c) {
        plataforma.exibirComponente(c, "#1A1A1A", "#F0F0F0");
    }
}

```

2.3 Composite

💡 Intenção: Compõe objetos em estruturas de árvore para representar hierarquias parte-todo. Permite que clientes tratem objetos individuais e composições de objetos de maneira uniforme.

Exemplo 1: Sistema de Arquivos (Pastas e Arquivos)

Um gerenciador de arquivos precisa calcular o tamanho de qualquer item, seja um arquivo simples ou uma pasta com subpastas. Composite trata ambos de forma uniforme.

```

public interface ItemSistemaArquivos {
    String getNome();
    long getTamanhoBytes();
    void exibir(int nivel);
}

public class Arquivo implements ItemSistemaArquivos {
    private final String nome;
    private final long tamanho;
    public Arquivo(String nome, long tamanho) {
        this.nome = nome; this.tamanho = tamanho;
    }
    public String getNome() { return nome; }
}

```

```

        public long getTamanhoBytes() { return tamanho; }
        public void exibir(int n) {
            System.out.println(" ".repeat(n*2) + "- " + nome + " (" + tamanho + "B)");
        }
    }

    public class Pasta implements ItemSistemaArquivos {
        private final String nome;
        private final List<ItemSistemaArquivos> filhos = new ArrayList<>();

        public Pasta(String nome) { this.nome = nome; }
        public void adicionar(ItemSistemaArquivos i) { filhos.add(i); }
        public String getNome() { return nome; }
        public long getTamanhoBytes() {
            return filhos.stream().mapToLong(ItemSistemaArquivos::getTamanhoBytes).sum();
        }
        public void exibir(int n) {
            System.out.println(" ".repeat(n*2) + "[+] " + nome);
            filhos.forEach(f -> f.exibir(n + 1));
        }
    }
}

```

Exemplo 2: Menu Hierárquico de Aplicação

Um sistema ERP possui menus com itens simples e submenus. Composite permite percorrer e renderizar toda a árvore de menus sem distinguir se um item é folha ou nó.

```

public interface ItemMenu {
    String getTitulo();
    void renderizar(int profundidade);
}

public class ItemMenuSimples implements ItemMenu {
    private final String titulo, url;
    public ItemMenuSimples(String titulo, String url) {
        this.titulo = titulo; this.url = url;
    }
    public String getTitulo() { return titulo; }
    public void renderizar(int p) {
        System.out.println(" ".repeat(p*4) + titulo + " -> " + url);
    }
}

public class SubMenu implements ItemMenu {
    private final String titulo;
    private final List<ItemMenu> filhos = new ArrayList<>();
    public SubMenu(String titulo) { this.titulo = titulo; }
    public void adicionar(ItemMenu item) { filhos.add(item); }
    public String getTitulo() { return titulo; }
    public void renderizar(int p) {
        System.out.println(" ".repeat(p*4) + "[" + titulo + "]");
        filhos.forEach(f -> f.renderizar(p + 1));
    }
}

// Uso:
SubMenu root = new SubMenu("Sistema");
SubMenu financeiro = new SubMenu("Financeiro");
financeiro.adicionar(new ItemMenuSimples("Contas a Pagar", "/fin/pagar"));
financeiro.adicionar(new ItemMenuSimples("Contas a Receber", "/fin/receber"));
root.adicionar(financeiro);
root.renderizar(0);

```

Exemplo 3: Cálculo de Imposto em Pedido com Itens e Kits

Um e-commerce vende produtos simples e kits (conjuntos de produtos). O cálculo de imposto deve funcionar igualmente para ambos. Composite permite somar impostos recursivamente.

```
public interface ProdutoTributavel {
    String getDescricao();
    double calcularImposto();
}

public class ProdutoSimples implements ProdutoTributavel {
    private final String descricao;
    private final double preco, aliquota;

    public ProdutoSimples(String d, double p, double a) {
        descricao = d; preco = p; aliquota = a;
    }
    public String getDescricao() { return descricao; }
    public double calcularImposto() { return preco * aliquota; }
}

public class KitProdutos implements ProdutoTributavel {
    private final String descricao;
    private final List<ProdutoTributavel> componentes = new ArrayList<>();

    public KitProdutos(String d) { descricao = d; }
    public void adicionar(ProdutoTributavel p) { componentes.add(p); }
    public String getDescricao() { return descricao; }
    public double calcularImposto() {
        return componentes.stream()
            .mapToDouble(ProdutoTributavel::calcularImposto)
            .sum();
    }
}
```

2.4 Decorator

 **Intenção:** Anexa responsabilidades adicionais a um objeto dinamicamente. Os Decoradores fornecem uma alternativa flexível ao uso de subclasses para extensão de funcionalidade.

Exemplo 1: Compressão e Criptografia de Arquivo

Um sistema de backup precisa salvar arquivos com diferentes combinações de compressão e criptografia. Decorator empilha comportamentos sem criar subclasses para cada combinação.

```
public interface ArquivoWriter {
    void escrever(byte[] dados);
}

public class ArquivoWriterBase implements ArquivoWriter {
    private final String caminho;
    public ArquivoWriterBase(String caminho) { this.caminho = caminho; }
    public void escrever(byte[] dados) {
        System.out.println("Gravando " + dados.length + " bytes em " + caminho);
    }
}

public abstract class WriterDecorator implements ArquivoWriter {
    protected final ArquivoWriter wrapped;
    public WriterDecorator(ArquivoWriter w) { this.wrapped = w; }
}

public class CompressaoDecorator extends WriterDecorator {
```

```

    public CompressaoDecorator(ArquivoWriter w) { super(w); }
    public void escrever(byte[] dados) {
        byte[] comprimido = comprimir(dados);
        System.out.println("Comprimindo: " + dados.length + " -> " + comprimido.length);
        wrapped.escrever(comprimido);
    }
    private byte[] comprimir(byte[] d) { return d; } // simplificado
}

public class CriptografiaDecorator extends WriterDecorator {
    public CriptografiaDecorator(ArquivoWriter w) { super(w); }
    public void escrever(byte[] dados) {
        System.out.println("Criptografando dados...");
        wrapped.escrever(criptografar(dados));
    }
    private byte[] criptografar(byte[] d) { return d; } // simplificado
}

// Uso (empilhando decoradores):
ArquivoWriter w = new CriptografiaDecorator(
    new CompressaoDecorator(
        new ArquivoWriterBase("/backup/dados.bkp")));
w.escrever(conteudo);

```

Exemplo 2: Validação em Cadeia de Campos de Formulário

Um formulário de cadastro precisa aplicar múltiplas validações (obrigatoriedade, tamanho mínimo, formato) a cada campo. Decorator encadeia validadores de forma flexível e reusável.

```

public interface Validador {
    boolean validar(String valor);
    String getMensagem();
}

public class ValidadorBase implements Validador {
    public boolean validar(String v) { return true; }
    public String getMensagem() { return "OK"; }
}

public abstract class ValidadorDecorator implements Validador {
    protected final Validador wrapped;
    private String mensagem = "OK";
    public ValidadorDecorator(Validador w) { this.wrapped = w; }
    public String getMensagem() { return mensagem; }
    protected void setMensagem(String m) { mensagem = m; }
}

public class ValidadorObrigatorio extends ValidadorDecorator {
    public ValidadorObrigatorio(Validador w) { super(w); }
    public boolean validar(String v) {
        if (v == null || v.isBlank()) {
            setMensagem("Campo obrigatório"); return false;
        }
        return wrapped.validar(v);
    }
}

public class ValidadorTamanhoMinimo extends ValidadorDecorator {
    private final int min;
    public ValidadorTamanhoMinimo(Validador w, int min) { super(w); this.min = min; }
    public boolean validar(String v) {
        if (v != null && v.length() < min) {
            setMensagem("Mínimo " + min + " caracteres"); return false;
        }
        return wrapped.validar(v);
    }
}

```

```

}

// Uso:
Validador v = new ValidadorTamanhoMinimo(
    new ValidadorObrigatorio(new ValidadorBase()), 8);
boolean ok = v.validar("abc"); // false: menos que 8 chars

```

Exemplo 3: Formatação de Saída de Texto em Console

Uma ferramenta de CLI precisa exibir mensagens com diferentes formatações (negrito, colorido, com prefixo de nível). Decorator aplica formatações progressivas sem modificar o código de saída base.

```

public interface Printer {
    String formatar(String texto);
}

public class SimplePrinter implements Printer {
    public String formatar(String texto) { return texto; }
}

public class NegritoPrinter implements Printer {
    private final Printer wrapped;
    public NegritoPrinter(Printer w) { this.wrapped = w; }
    public String formatar(String t) {
        return "\033[1m" + wrapped.formatar(t) + "\033[0m";
    }
}

public class ColorPrinter implements Printer {
    private final Printer wrapped;
    private final String codigoANSI;
    public ColorPrinter(Printer w, String codigo) {
        this.wrapped = w; this.codigoANSI = codigo;
    }
    public String formatar(String t) {
        return codigoANSI + wrapped.formatar(t) + "\033[0m";
    }
}

public class PrefixoPrinter implements Printer {
    private final Printer wrapped;
    private final String prefixo;
    public PrefixoPrinter(Printer w, String p) { this.wrapped=w; this.prefixo=p; }
    public String formatar(String t) {
        return prefixo + " " + wrapped.formatar(t);
    }
}

// Uso:
Printer p = new NegritoPrinter(
    new ColorPrinter(
        new PrefixoPrinter(new SimplePrinter(), "[ERRO]"),
        "\033[31m")); // vermelho
System.out.println(p.formatar("Arquivo não encontrado!"));

```

2.5 Facade

 *Intenção: Fornece uma interface simplificada para um conjunto de interfaces em um subsistema. Define uma interface de alto nível que torna o subsistema mais fácil de usar.*

Exemplo 1: Facade para Processo de Checkout

O processo de finalização de compra envolve validação de estoque, cálculo de frete, processamento de pagamento e envio de e-mail de confirmação. A Facade unifica todas essas operações em uma única chamada.

```
// Subsistemas complexos
class ServicoEstoque {
    public boolean verificar(String produtoId, int qtd) {
        System.out.println("Verificando estoque..."); return true;
    }
    public void reservar(String produtoId, int qtd) {
        System.out.println("Estoque reservado.");
    }
}
class ServicoFrete {
    public double calcular(String cep) {
        System.out.println("Calculando frete para " + cep); return 15.0;
    }
}
class ServicoPagamento {
    public boolean processar(String cartao, double valor) {
        System.out.println("Processando pagamento R$" + valor); return true;
    }
}
class ServicoEmail {
    public void enviarConfirmacao(String email, String pedidoId) {
        System.out.println("Email enviado para " + email);
    }
}

// Facade
public class CheckoutFacade {
    private final ServicoEstoque estoque = new ServicoEstoque();
    private final ServicoFrete frete = new ServicoFrete();
    private final ServicoPagamento pag = new ServicoPagamento();
    private final ServicoEmail email = new ServicoEmail();

    public boolean finalizarCompra(String prodId, int qtd,
                                   String cep, String cartao,
                                   double valor, String emailCliente) {
        if (!estoque.verificar(prodId, qtd)) return false;
        valor += frete.calcular(cep);
        if (!pag.processar(cartao, valor)) return false;
        estoque.reservar(prodId, qtd);
        email.enviarConfirmacao(emailCliente, "PED-" + System.currentTimeMillis());
        return true;
    }
}

// Uso com uma única chamada:
new CheckoutFacade().finalizarCompra(
    "PROD-1", 2, "74000000", "4111...", 200.0, "c@email.com");
```

Exemplo 2: Facade para Conversão e Upload de Vídeo

Uma plataforma de ensino online precisa, ao receber um vídeo, convertê-lo para múltiplos formatos, gerar thumbnail, e fazer upload para CDN. A Facade encapsula toda essa complexidade.

```
class ConversorVideo {
    public String converter(String arquivo, String formato) {
        System.out.println("Convertendo para " + formato); return arquivo + "." +
        formato;
    }
}
class GeradorThumbnail {
    public String gerar(String video) {
        System.out.println("Gerando thumbnail..."); return "thumb_" + video + ".jpg";
    }
}
```

```

    }
}
class ClienteCDN {
    public String upload(String arquivo) {
        System.out.println("Enviando " + arquivo + " para CDN");
        return "https://cdn.puc.br/" + arquivo;
    }
}

public class VideoProcessingFacade {
    private final ConversorVideo conversor = new ConversorVideo();
    private final GeradorThumbnail thumb = new GeradorThumbnail();
    private final ClienteCDN cdn = new ClienteCDN();

    public Map<String, String> processar(String videoOriginal) {
        Map<String, String> urls = new HashMap<>();
        for (String fmt : List.of("mp4", "webm")) {
            String convertido = conversor.converter(videoOriginal, fmt);
            urls.put(fmt, cdn.upload(convertido));
        }
        urls.put("thumb", cdn.upload(thumb.gerar(videoOriginal)));
        return urls;
    }
}

```

Exemplo 3: Facade para Autenticação e Autorização

Um sistema corporativo realiza autenticação via LDAP, verifica permissões em um banco de dados e registra o acesso em log de auditoria. A Facade oferece uma API simples de autenticação.

```

class ServicoLDAP {
    public boolean autenticar(String user, String senha) {
        System.out.println("Autenticando no LDAP: " + user);
        return !senha.isEmpty();
    }
}

class ServicoPermissoes {
    public Set<String> obterPermissoes(String user) {
        System.out.println("Buscando permissões de " + user);
        return Set.of("READ", "WRITE");
    }
}

class AuditLog {
    public void registrarAcesso(String user, boolean sucesso) {
        System.out.println("Auditoria: " + user + " = " + sucesso);
    }
}

public class AuthFacade {
    private final ServicoLDAP ldap = new ServicoLDAP();
    private final ServicoPermissoes perms = new ServicoPermissoes();
    private final AuditLog audit = new AuditLog();

    public Optional<Set<String>> login(String user, String senha) {
        boolean ok = ldap.autenticar(user, senha);
        audit.registrarAcesso(user, ok);
        if (!ok) return Optional.empty();
        return Optional.of(perms.obterPermissoes(user));
    }
}

// Uso:
Optional<Set<String>> perms = new AuthFacade().login("joao", "senha123");
perms.ifPresent(p -> System.out.println("Permissões: " + p));

```

2.6 Flyweight

 *Intenção: Usa compartilhamento para suportar eficientemente grandes quantidades de objetos de granularidade fina.*

Exemplo 1: Renderização de Caracteres em Editor de Texto

Um editor de texto com milhões de caracteres armazena apenas o estilo (cor, fonte, tamanho) de cada caractere, compartilhando o objeto de estilo entre todos os caracteres com as mesmas propriedades.

```
// Estado intrínseco (compartilhado)
public class EstiloCaractere {
    private final String fonte;
    private final int tamanho;
    private final String cor;

    public EstiloCaractere(String fonte, int tam, String cor) {
        this.fonte = fonte; this.tamanho = tam; this.cor = cor;
    }

    public void renderizar(char c, int x, int y) {
        System.out.printf("%s '%c' em (%d,%d) [%s %dpt %s]%n",
            "Renderizando", c, x, y, fonte, tamanho, cor);
    }
}

// Flyweight Factory
public class EstiloFactory {
    private static final Map<String, EstiloCaractere> cache = new HashMap<>();

    public static EstiloCaractere getEstilo(String fonte, int tam, String cor) {
        String chave = fonte + "-" + tam + "-" + cor;
        return cache.computeIfAbsent(chave,
            k -> new EstiloCaractere(fonte, tam, cor));
    }

    public static int getTotalCriados() { return cache.size(); }
}

// Uso (cada Caractere usa estado extrínseco: posição/valor)
for (int i = 0; i < 10000; i++) {
    EstiloCaractere estilo = EstiloFactory.getEstilo("Arial", 12, "#000");
    estilo.renderizar('A', i % 80, i / 80);
}

System.out.println("Objetos criados: " + EstiloFactory.getTotalCriados()); // = 1
```

Exemplo 2: Pool de Conexões com Banco de Dados

Criar conexões com banco é custoso. O Flyweight compartilha conexões já abertas entre diferentes partes do sistema, evitando overhead de criação repetida.

```
public class ConexaoFlyweight {
    private final String url;
    private boolean emUso = false;

    public ConexaoFlyweight(String url) { this.url = url; }
    public boolean isEmUso() { return emUso; }
    public void adquirir() { emUso = true; }
    public void liberar() { emUso = false; }
    public void executar(String sql) {
        System.out.println "[" + url + " ] " + sql;
    }
}

public class PoolConexoes {
```

```

private final List<ConexaoFlyweight> pool = new ArrayList<>();
private static final int TAMANHO = 5;

public PoolConexoes(String url) {
    for (int i = 0; i < TAMANHO; i++)
        pool.add(new ConexaoFlyweight(url));
}

public ConexaoFlyweight adquirir() {
    return pool.stream()
        .filter(c -> !c.isEmUso())
        .findFirst()
        .map(c -> { c.adquirir(); return c; })
        .orElseThrow(() -> new RuntimeException("Pool esgotado"));
}
}

// Uso:
PoolConexoes pool = new PoolConexoes("jdbc:pg://localhost/db");
ConexaoFlyweight conn = pool.adquirir();
conn.executar("SELECT * FROM alunos");
conn.liberar();

```

Exemplo 3: Ícones Compartilhados em Interface Gráfica

Um sistema de gestão exibe milhares de ícones em uma lista de arquivos. Ao invés de criar um objeto de ícone para cada item, o Flyweight compartilha o objeto de ícone por tipo de arquivo.

```

// Estado intrínseco: imagem do ícone
public class Icone {
    private final String tipo;
    private final byte[] imagemPNG; // dados compartilhados

    public Icone(String tipo) {
        this.tipo = tipo;
        this.imagemPNG = carregarImagem(tipo); // operação custosa
        System.out.println("Ícone carregado: " + tipo);
    }
    private byte[] carregarImagem(String t) { return new byte[1024]; }

    // Estado extrínseco (posição) passado na chamada
    public void desenhar(int x, int y, String nomeArquivo) {
        System.out.printf("%s (%d,%d) %s\n", tipo, x, y, nomeArquivo);
    }
}

public class IconeFactory {
    private static final Map<String, Icone> icones = new HashMap<>();

    public static Icone get(String extensao) {
        return icones.computeIfAbsent(extensao, Icone::new);
    }
}

// Uso com 50.000 arquivos - apenas ~20 ícones criados (um por extensão):
for (String arquivo : listaArquivos) {
    String ext = arquivo.substring(arquivo.lastIndexOf('.')+1);
    IconeFactory.get(ext).desenhar(10, 10 * i, arquivo);
}

```

💡 *Intenção: Fornece um objeto representante (substituto) de outro objeto para controlar o acesso a ele.*

Exemplo 1: Proxy de Cache para Consulta de Relatório

Consultas a relatórios complexos são demoradas. O Proxy Cache armazena o resultado da primeira consulta e o retorna nas chamadas seguintes, sem acessar o banco novamente.

```
public interface ServicoRelatorio {
    String gerarRelatorio(String filtro);
}

public class ServicoRelatorioReal implements ServicoRelatorio {
    public String gerarRelatorio(String filtro) {
        System.out.println("[BD] Gerando relatório para: " + filtro);
        try { Thread.sleep(2000); } catch (Exception e) {}
        return "Relatório-" + filtro;
    }
}

public class ProxyCache implements ServicoRelatorio {
    private final ServicoRelatorio real = new ServicoRelatorioReal();
    private final Map<String, String> cache = new HashMap<>();

    public String gerarRelatorio(String filtro) {
        if (cache.containsKey(filtro)) {
            System.out.println("[Cache] Retornando do cache: " + filtro);
            return cache.get(filtro);
        }
        String resultado = real.gerarRelatorio(filtro);
        cache.put(filtro, resultado);
        return resultado;
    }
}

// Uso:
ServicoRelatorio svc = new ProxyCache();
svc.gerarRelatorio("jan-2025"); // acessa BD
svc.gerarRelatorio("jan-2025"); // retorna do cache
```

Exemplo 2: Proxy de Controle de Acesso

Um repositório de documentos sigilosos só pode ser acessado por usuários com perfil 'ADMIN'. O Proxy de Proteção verifica as permissões antes de delegar ao objeto real.

```
public interface DocumentoRepositorio {
    String obter(String docId);
    void salvar(String docId, String conteudo);
}

public class DocumentoRepositorioImpl implements DocumentoRepositorio {
    private final Map<String, String> docs = new HashMap<>();
    public String obter(String id) { return docs.getDefault(id, ""); }
    public void salvar(String id, String cont) { docs.put(id, cont); }
}

public class ProxyProtecao implements DocumentoRepositorio {
    private final DocumentoRepositorio real = new DocumentoRepositorioImpl();
    private final Usuario usuarioAtual;

    public ProxyProtecao(Usuario u) { this.usuarioAtual = u; }

    public String obter(String id) {
        verificarPermissao("READ");
        return real.obter(id);
    }
}
```

```
    }
    public void salvar(String id, String cont) {
        verificarPermissao("WRITE");
        real.salvar(id, cont);
    }
    private void verificarPermissao(String op) {
        if (!usuarioAtual.temPermissao(op))
            throw new SecurityException("Sem permissão para: " + op);
    }
}
```

Exemplo 3: Proxy Virtual para Carregamento Lazy de Imagem

Em uma galeria de fotos, cada imagem é carregada da rede somente quando o usuário rolar até ela. O Proxy Virtual adia o carregamento custoso até o momento necessário.

```
public interface Imagem {
    void exibir();
    int getLargura();
    int getAltura();
}

public class ImagemReal implements Imagem {
    private final String url;
    private byte[] dados;

    public ImagemReal(String url) {
        this.url = url;
        System.out.println("[Rede] Baixando: " + url); // operação custosa
        this.dados = new byte[1024]; // simula download
    }
    public void exibir() { System.out.println("Exibindo: " + url); }
    public int getLargura() { return 1920; }
    public int getAltura() { return 1080; }
}

public class ProxyLazy implements Imagem {
    private final String url;
    private ImagemReal imagemReal; // null até ser necessário

    public ProxyLazy(String url) { this.url = url; }

    private ImagemReal getOuCarregar() {
        if (imagemReal == null) imagemReal = new ImagemReal(url);
        return imagemReal;
    }

    public void exibir() { getOuCarregar().exibir(); }
    public int getLargura() { return getOuCarregar().getLargura(); }
    public int getAltura() { return getOuCarregar().getAltura(); }
}
```

Padrões Comportamentais (Behavioral Patterns)

3.1 Chain of Responsibility

💡 *Intenção: Evita o acoplamento do remetente de uma solicitação ao seu receptor, ao dar a mais de um objeto a oportunidade de tratar a solicitação. Encadeia os objetos receptores e passa a solicitação ao longo da cadeia.*

Exemplo 1: Pipeline de Validação de Formulário

O cadastro de um usuário passa por etapas: validar formato de e-mail, verificar força da senha, checar duplicidade no banco. Cada handler processa sua parte e passa para o próximo.

```
public abstract class ValidadorHandler {
    private ValidadorHandler proximo;

    public ValidadorHandler setProximo(ValidadorHandler p) {
        this.proximo = p; return p;
    }
    protected void passarParaProximo(String campo, String valor) {
        if (proximo != null) proximo.validar(campo, valor);
    }
    public abstract void validar(String campo, String valor);
}

public class ValidadorEmail extends ValidadorHandler {
    public void validar(String campo, String valor) {
        if (!valor.contains("@"))
            throw new IllegalArgumentException("E-mail inválido");
        passarParaProximo(campo, valor);
    }
}

public class ValidadorSenhaDificuldade extends ValidadorHandler {
    public void validar(String campo, String valor) {
        if (valor.length() < 8)
            throw new IllegalArgumentException("Senha muito curta");
        passarParaProximo(campo, valor);
    }
}

// Montagem da cadeia:
ValidadorHandler cadeia = new ValidadorEmail();
cadeia.setProximo(new ValidadorSenhaDificuldade());

cadeia.validar("email", "user@puc.br");
```

Exemplo 2: Escalada de Suporte Técnico (N1 → N2 → N3)

Chamados de suporte são triados pelo atendimento N1; se não resolvidos, sobem para N2 e depois N3 (especialista). Cada nível resolve o que pode e escala o restante.

```
public abstract class Suporte {
    protected Suporte superior;
    public void setSuperior(Suporte s) { this.superior = s; }
    public abstract void atender Chamado c;
    protected void escalar(Chamado c) {
        if (superior != null) superior.atender(c);
        else System.out.println("Sem solução: " + c.getDescricao());
    }
}
```

```

public class SuporteN1 extends Suporte {
    public void atender(Chamado c) {
        if (c.getNivel() <= 1) {
            System.out.println("N1 resolveu: " + c.getDescricao());
        } else { System.out.println("N1 escalando..."); escalar(c); }
    }
}

public class SuporteN2 extends Suporte {
    public void atender(Chamado c) {
        if (c.getNivel() <= 2) {
            System.out.println("N2 resolveu: " + c.getDescricao());
        } else { System.out.println("N2 escalando..."); escalar(c); }
    }
}

public class SuporteN3 extends Suporte {
    public void atender(Chamado c) {
        System.out.println("N3 (especialista) resolveu: " + c.getDescricao());
    }
}

// Montagem:
Suporte n1 = new SuporteN1(), n2 = new SuporteN2(), n3 = new SuporteN3();
n1.setSuperior(n2); n2.setSuperior(n3);
n1.atender(new Chamado(3, "Falha no servidor de produção"));

```

Exemplo 3: Filtros de Segurança em Requisição HTTP

Uma API REST aplica filtros sequenciais em cada requisição: autenticação JWT, rate limiting, sanitização de entrada e logging. Cada filtro na cadeia processa a requisição e decide se continua.

```

public abstract class FiltroHTTP {
    private FiltroHTTP proximo;
    public FiltroHTTP setProximo(FiltroHTTP p) { proximo = p; return p; }
    public abstract boolean processar(Requisicao req, Resposta resp);
    protected boolean prosseguir(Requisicao req, Resposta resp) {
        return proximo == null || proximo.processar(req, resp);
    }
}

public class FiltroAutenticacao extends FiltroHTTP {
    public boolean processar(Requisicao req, Resposta resp) {
        if (req.getHeader("Authorization") == null) {
            resp.setStatus(401); return false;
        }
        return prosseguir(req, resp);
    }
}

public class FiltroRateLimit extends FiltroHTTP {
    private final Map<String, Integer> contadores = new HashMap<>();
    public boolean processar(Requisicao req, Resposta resp) {
        int count = contadores.merge(req.getIp(), 1, Integer::sum);
        if (count > 100) { resp.setStatus(429); return false; }
        return prosseguir(req, resp);
    }
}

// Montagem do pipeline:
FiltroHTTP pipeline = new FiltroAutenticacao();
pipeline.setProximo(new FiltroRateLimit());

```

3.2 Command

💡 *Intenção: Encapsula uma solicitação como um objeto, permitindo parametrizar clientes com diferentes solicitações, enfileirar ou registrar solicitações e suportar operações que podem ser desfeitas.*

Exemplo 1: Histórico de Ações com Undo/Redo em Editor

Um editor de texto com suporte a desfazer/refazer encapsula cada ação (inserir texto, deletar) como um objeto Command. A pilha de histórico permite reverter ou reaplicar operações.

```
public interface Comando {
    void executar();
    void desfazer();
}

public class ComandoInserirTexto implements Comando {
    private final StringBuilder documento;
    private final int posicao;
    private final String texto;

    public ComandoInserirTexto(StringBuilder doc, int pos, String txt) {
        documento = doc; posicao = pos; texto = txt;
    }

    public void executar() { documento.insert(posicao, texto); }
    public void desfazer() { documento.delete(posicao, posicao + texto.length()); }
}

public class Historico {
    private final Deque<Comando> historico = new ArrayDeque<>();
    private final Deque<Comando> refazer = new ArrayDeque<>();

    public void executar(Comando c) {
        c.executar(); historico.push(c); refazer.clear();
    }

    public void undo() {
        if (!historico.isEmpty()) {
            Comando c = historico.pop(); c.desfazer(); refazer.push(c);
        }
    }

    public void redo() {
        if (!refazer.isEmpty()) {
            Comando c = refazer.pop(); c.executar(); historico.push(c);
        }
    }
}
```

Exemplo 2: Fila de Tarefas para Processamento Assíncrono

Um sistema de processamento de imagens recebe requisições que são enfileiradas e processadas por workers. Command permite encapsular cada tarefa e agendar sua execução.

```
public interface TarefaCommand {
    void executar();
    String getDescricao();
}

public class TarefaRedimensionarImagem implements TarefaCommand {
    private final String imagemPath;
    private final int largura, altura;

    public TarefaRedimensionarImagem(String path, int l, int a) {
        imagemPath = path; largura = l; altura = a;
    }

    public void executar() {
        System.out.printf("Redimensionando %s para %dx%d\n",
            imagemPath, largura, altura);
    }
}
```

```

    public String getDescricao() { return "Redimensionar: " + imagemPath; }
}

public class FilaProcessamento {
    private final Queue<TarefaCommand> fila = new LinkedList<>();

    public void enqueue(TarefaCommand t) { fila.offer(t); }

    public void processarTodas() {
        while (!fila.isEmpty()) {
            TarefaCommand t = fila.poll();
            System.out.println("Processando: " + t.getDescricao());
            t.executar();
        }
    }
}

```

Exemplo 3: Sistema de Macro em Planilha

Uma planilha eletrônica permite gravar e reproduzir macros (sequências de ações). Command encapsula cada ação e a macro é uma lista de comandos que pode ser executada repetidamente.

```

public interface AcaoPlanilha {
    void executar(Planilha p);
}

public class AcaoAlterarCelula implements AcaoPlanilha {
    private final String celula; private final String valor;
    public AcaoAlterarCelula(String c, String v) { celula=c; valor=v; }
    public void executar(Planilha p) { p.setCelula(celula, valor); }
}

public class AcaoFormatarCelula implements AcaoPlanilha {
    private final String celula; private final String estilo;
    public AcaoFormatarCelula(String c, String e) { celula=c; estilo=e; }
    public void executar(Planilha p) { p.setEstilo(celula, estilo); }
}

public class Macro {
    private final String nome;
    private final List<AcaoPlanilha> acoes = new ArrayList<>();

    public Macro(String nome) { this.nome = nome; }
    public void addAcao(AcaoPlanilha a) { acoes.add(a); }

    public void executar(Planilha p) {
        System.out.println("Executando macro: " + nome);
        acoes.forEach(a -> a.executar(p));
    }
}

// Uso:
Macro macro = new Macro("Formatar Cabeçalho");
macro.addAcao(new AcaoAlterarCelula("A1", "Nome"));
macro.addAcao(new AcaoFormatarCelula("A1", "BOLD"));
macro.executar(planilha);

```

3.3 Iterator

💡 *Intenção: Fornece uma maneira de acessar sequencialmente os elementos de um objeto agregado sem expor sua representação interna.*

Exemplo 1: Iterador de Árvore de Categorias

Um sistema de e-commerce armazena categorias em árvore. O Iterador permite percorrer todos os nós em ordem (DFS) sem expor a estrutura interna da árvore ao código cliente.

```
public class Categoria {
    private final String nome;
    private final List<Categoria> subcategorias;

    public Categoria(String nome) {
        this.nome = nome; this.subcategorias = new ArrayList<>();
    }
    public void adicionar(Categoria c) { subcategorias.add(c); }
    public String getNome() { return nome; }
    public List<Categoria> getSubs() { return subcategorias; }

    public Iterator<Categoria> iteradorDFS() {
        return new DFSIterator(this);
    }
}

class DFSIterator implements Iterator<Categoria> {
    private final Deque<Categoria> pilha = new ArrayDeque<>();

    public DFSIterator(Categoria raiz) { pilha.push(raiz); }

    public boolean hasNext() { return !pilha.isEmpty(); }

    public Categoria next() {
        Categoria atual = pilha.pop();
        List<Categoria> subs = new ArrayList<>(atual.getSubs());
        Collections.reverse(subs);
        subs.forEach(pilha::push);
        return atual;
    }
}

// Uso:
Iterator<Categoria> it = raiz.iteradorDFS();
while (it.hasNext()) {
    System.out.println(it.next().getNome());
}
```

Exemplo 2: Iterador de Registros de CSV em Lote

Um importador de dados precisa ler um arquivo CSV grande linha por linha sem carregá-lo inteiramente em memória. O Iterador faz leitura lazy de cada linha.

```
public class CsvIterator implements Iterator<String[]>, Closeable {
    private final BufferedReader reader;
    private String proximaLinha;
    private final String separador;

    public CsvIterator(String caminho, String sep) throws IOException {
        reader = new BufferedReader(new FileReader(caminho));
        separador = sep;
        reader.readLine(); // pular cabeçalho
        proximaLinha = reader.readLine();
    }

    public boolean hasNext() { return proximaLinha != null; }

    public String[] next() {
        if (!hasNext()) throw new NoSuchElementException();
        String[] campos = proximaLinha.split(separador);
    }
}
```

```

        try { proximaLinha = reader.readLine(); }
        catch (IOException e) { proximaLinha = null; }
        return campos;
    }

    public void close() throws IOException { reader.close(); }
}

// Uso:
try (CsvIterator it = new CsvIterator("alunos.csv", ";")) {
    while (it.hasNext()) {
        String[] linha = it.next();
        System.out.println("RA: " + linha[0] + " Nome: " + linha[1]);
    }
}

```

Exemplo 3: Iterador de Paginação de API REST

Uma API retorna resultados paginados. O Iterador abstrai a lógica de paginação, permitindo que o código cliente percorra todos os itens como se fossem uma coleção única.

```

public class PaginacaoIterator<T> implements Iterator<List<T>> {
    private final Function<Integer, List<T>> buscador;
    private int paginaAtual = 1;
    private List<T> buffer;
    private final int tamPagina;

    public PaginacaoIterator(Function<Integer, List<T>> b, int tam) {
        buscador = b; tamPagina = tam;
        buffer = buscador.apply(paginaAtual);
    }

    public boolean hasNext() { return !buffer.isEmpty(); }

    public List<T> next() {
        List<T> pagina = new ArrayList<>(buffer);
        paginaAtual++;
        buffer = buscador.apply(paginaAtual);
        return pagina;
    }
}

// Uso simulando uma API de alunos:
PaginacaoIterator<Aluno> paginador = new PaginacaoIterator<>(
    pagina -> alunoRepository.buscarPagina(pagina, 20), 20);

while (paginador.hasNext()) {
    List<Aluno> pagina = paginador.next();
    pagina.forEach(a -> System.out.println(a.getNome()));
}

```

3.4 Mediator

 *Intenção: Define um objeto que encapsula a forma como um conjunto de objetos interage. O Mediator promove o fraco acoplamento ao evitar que os objetos se refiram explicitamente uns aos outros.*

Exemplo 1: Chat em Sala de Aula Virtual

Numa sala virtual de EAD, professores e alunos se comunicam via chat. Cada participante não envia mensagens diretamente aos outros — todas passam pelo Mediator (sala), que as distribui.

```
public interface Participante {
    void receberMensagem(String remetente, String msg);
    String getNome();
}

public class SalaVirtual {
    private final List<Participante> participantes = new ArrayList<>();

    public void entrar(Participante p) { participantes.add(p); }

    public void enviarMensagem(Participante remetente, String msg) {
        participantes.stream()
            .filter(p -> p != remetente)
            .forEach(p -> p.receberMensagem(remetente.getNome(), msg));
    }
}

public class Usuario implements Participante {
    private final String nome;
    private final SalaVirtual sala;

    public Usuario(String nome, SalaVirtual sala) {
        this.nome = nome; this.sala = sala; sala.entrar(this);
    }
    public void enviar(String msg) { sala.enviarMensagem(this, msg); }
    public void receberMensagem(String rem, String msg) {
        System.out.println(nome + " recebeu de " + rem + ": " + msg);
    }
    public String getNome() { return nome; }
}

// Uso:
SalaVirtual sala = new SalaVirtual();
Usuario prof = new Usuario("Prof. Carlos", sala);
Usuario al1 = new Usuario("Ana", sala);
Usuario al2 = new Usuario("Pedro", sala);
prof.enviar("Bem-vindos à aula de Design Patterns!");
```

Exemplo 2: Coordenação de Componentes em Formulário Complexo

Um formulário de pedido tem campos interdependentes: ao selecionar um estado, a lista de cidades é atualizada; ao selecionar frete expresso, a data estimada muda. O Mediator coordena essas interações.

```
public class FormularioPedidoMediator {
    private ComboBox cmbEstado, cmbCidade;
    private RadioButton rbFreteNormal, rbFreteExpresso;
    private Label lblDataEstimada;

    public void setComponentes(ComboBox estado, ComboBox cidade,
                               RadioButton normal, RadioButton expresso,
                               Label dataLabel) {
        cmbEstado = estado; cmbCidade = cidade;
        rbFreteNormal = normal; rbFreteExpresso = expresso;
        lblDataEstimada = dataLabel;
    }

    public void onEstadoAlterado(String novoEstado) {
        List<String> cidades = buscarCidades(novoEstado);
        cmbCidade.setOpcoes(cidades);
        recalcularEntrega();
    }

    public void onFreteAlterado(boolean expresso) {
        recalcularEntrega();
    }
}
```

```

    }

    private void recalcularEntrega() {
        int dias = rbFreteExpresso.isSelected() ? 1 : 5;
        lblDataEstimada.setText(
            LocalDate.now().plusDays(dias).toString());
    }
    private List<String> buscarCidades(String uf) {
        return List.of("Goiânia", "Anápolis"); // simplificado
    }
}

```

Exemplo 3: Controle de Tráfego Aéreo

Aeronaves não se comunicam diretamente entre si — toda comunicação passa pela torre de controle (Mediator), que coordena rotas, pistas e alertas de colisão.

```

public interface Aeronave {
    String getIdentificacao();
    void receberInstrucao(String instrucao);
}

public class TorreControle {
    private final List<Aeronave> aeronaves = new ArrayList<>();
    private final Set<String> pistasOcupadas = new HashSet<>();

    public void registrar(Aeronave a) { aeronaves.add(a); }

    public boolean solicitarPouso(Aeronave a, String pista) {
        if (pistasOcupadas.contains(pista)) {
            a.receberInstrucao("Aguarde, pista " + pista + " ocupada");
            return false;
        }
        pistasOcupadas.add(pista);
        a.receberInstrucao("Autorizado para pista " + pista);
        notificarOutras(a, "Pista " + pista + " em uso por " + a.getIdentificacao());
        return true;
    }

    public void liberarPista(String pista) { pistasOcupadas.remove(pista); }

    private void notificarOutras(Aeronave origem, String msg) {
        aeronaves.stream().filter(a -> a != origem)
            .forEach(a -> a.receberInstrucao(msg));
    }
}

```

3.5 Memento

💡 Intenção: Sem violar o encapsulamento, captura e externaliza o estado interno de um objeto para que o objeto possa ser restaurado para esse estado mais tarde.

Exemplo 1: Histórico de Versões de Documento

Um processador de texto salva snapshots do documento a cada alteração. O Memento encapsula cada versão e o Caretaker mantém o histórico, permitindo reverter para qualquer ponto anterior.

```

public class Documento {
    private String titulo, conteudo;
    private String ultimaAlteracao;

    public Documento(String titulo) { this.titulo = titulo; conteudo = ""; }
}

```

```

public void editar(String novoConteudo) {
    this.conteudo = novoConteudo;
    this.ultimaAlteracao = LocalDateTime.now().toString();
}

// Criação do Memento
public Memento salvar() {
    return new Memento(titulo, conteudo, ultimaAlteracao);
}

// Restauração a partir do Memento
public void restaurar(Memento m) {
    this.titulo = m.getTitulo();
    this.conteudo = m.getConteudo();
    this.ultimaAlteracao = m.getAlteracao();
}

public static class Memento {
    private final String titulo, conteudo, alteracao;
    private Memento(String t, String c, String a) {
        titulo = t; conteudo = c; alteracao = a;
    }
    String getTitulo() { return titulo; }
    String getConteudo() { return conteudo; }
    String getAlteracao() { return alteracao; }
}

}

public class HistoricoDocumento {
    private final Deque<Documento.Memento> versoes = new ArrayDeque<>();
    public void salvar(Documento d) { versoes.push(d.salvar()); }
    public void desfazer(Documento d) {
        if (!versoes.isEmpty()) d.restaurar(versoes.pop());
    }
}

```

Exemplo 2: Checkpoint em Jogo de Videogame

Um jogo RPG permite ao jogador salvar o progresso em checkpoints. O Memento armazena a posição, vida e inventário do personagem para restauração quando o jogador morrer.

```

public class EstadoJogo {
    private String nivelAtual;
    private int vidaJogador;
    private List<String> inventario;
    private double posX, posY;

    public EstadoJogo(String nivel, int vida, List<String> inv, double x, double y) {
        nivelAtual = nivel; vidaJogador = vida;
        inventario = new ArrayList<>(inv); posX = x; posY = y;
    }

    public Memento criarCheckpoint() {
        return new Memento(nivelAtual, vidaJogador, new ArrayList<>(inventario), posX,
posY);
    }
    public void restaurar(Memento m) {
        nivelAtual = m.nivel; vidaJogador = m.vida;
        inventario = new ArrayList<>(m.inventario); posX = m.x; posY = m.y;
    }

    public static class Memento {
        final String nivel; final int vida;
        final List<String> inventario; final double x, y;
    }
}

```

```

        Memento(String n, int v, List<String> i, double x, double y) {
            nivel = n; vida = v; inventario = i; this.x = x; this.y = y;
        }
    }
}

// Uso:
EstadoJogo jogo = new EstadoJogo("Floresta", 100, List.of("Espada"), 10, 20);
EstadoJogo.Memento checkpoint = jogo.criarCheckpoint();
// ... jogador morre ...
jogo.restaurar(checkpoint); // volta ao checkpoint

```

Exemplo 3: Rollback de Transação em Memória

Um sistema de transferência bancária precisa garantir que, em caso de falha, o estado das contas seja revertido. Memento salva o estado antes da transação para possibilitar rollback.

```

public class ContaBancaria {
    private final String numero;
    private double saldo;

    public ContaBancaria(String numero, double saldo) {
        this.numero = numero; this.saldo = saldo;
    }
    public void debitar(double v) { saldo -= v; }
    public void creditar(double v) { saldo += v; }
    public double getSaldo() { return saldo; }

    public Snapshot salvar() { return new Snapshot(saldo); }
    public void restaurar(Snapshot s) { saldo = s.saldo; }

    public static class Snapshot {
        private final double saldo;
        Snapshot(double s) { saldo = s; }
    }
}

public class TransacaoSegura {
    public void transferir(ContaBancaria origem, ContaBancaria destino, double v) {
        ContaBancaria.Snapshot snapOrigem = origem.salvar();
        ContaBancaria.Snapshot snapDestino = destino.salvar();
        try {
            origem.debitar(v);
            if (origem.getSaldo() < 0) throw new IllegalStateException("Saldo insuf.");
            destino.creditar(v);
            System.out.println("Transferência concluída: R$" + v);
        } catch (Exception e) {
            origem.restaurar(snapOrigem);
            destino.restaurar(snapDestino);
            System.out.println("Rollback executado: " + e.getMessage());
        }
    }
}

```

3.6 Observer

 **Intenção:** Define uma dependência um-para-muitos entre objetos, de modo que quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente.

Exemplo 1: Notificações de Variação de Preço de Ação

Num sistema de bolsa de valores, múltiplos investidores cadastram alertas para um determinado ativo. Quando o preço muda, todos os observadores registrados são notificados automaticamente.

```
public interface ObservadorPreco {
    void onPrecoAlterado(String ticker, double precoAnterior, double novoPreco);
}

public class AtivoFinanceiro {
    private final String ticker;
    private double precoAtual;
    private final List<ObservadorPreco> observadores = new ArrayList<>();

    public AtivoFinanceiro(String ticker, double precoInicial) {
        this.ticker = ticker; this.precoAtual = precoInicial;
    }
    public void registrar(ObservadorPreco o) { observadores.add(o); }
    public void cancelar(ObservadorPreco o) { observadores.remove(o); }

    public void setPreco(double novoPreco) {
        double anterior = precoAtual;
        precoAtual = novoPreco;
        observadores.forEach(o -> o.onPrecoAlterado(ticker, anterior, novoPreco));
    }
}

public class AlertaPrecoMinimo implements ObservadorPreco {
    private final String nomeInvestidor; private final double limiteMinimo;
    public AlertaPrecoMinimo(String nome, double limite) {
        nomeInvestidor = nome; limiteMinimo = limite;
    }
    public void onPrecoAlterado(String t, double ant, double novo) {
        if (novo < limiteMinimo)
            System.out.printf("%s: %s caiu abaixo de %.2f (atual: %.2f)\n",
                               nomeInvestidor, t, limiteMinimo, novo);
    }
}
```

Exemplo 2: Propagação de Eventos em Interface Reativa

Um carrinho de compras atualiza automaticamente o subtotal, o frete estimado e os cupons disponíveis sempre que o usuário adiciona ou remove um produto.

```
public interface CarrinhoObserver {
    void onCarrinhoAtualizado(List<Item> itens, double total);
}

public class Carrinho {
    private final List<Item> itens = new ArrayList<>();
    private final List<CarrinhoObserver> observers = new ArrayList<>();

    public void addObserver(CarrinhoObserver o) { observers.add(o); }
    public void removeObserver(CarrinhoObserver o) { observers.remove(o); }

    public void addItem(Item item) {
        itens.add(item); notificar();
    }
    public void removeItem(Item item) {
        itens.remove(item); notificar();
    }
    private void notificar() {
        double total = itens.stream().mapToDouble(Item::getPreco).sum();
        observers.forEach(o -> o.onCarrinhoAtualizado(itens, total));
    }
}
```

```
// Observadores:
Carrinho c = new Carrinho();
c.addObserver((itens, total) ->
    System.out.printf("Subtotal: R$ %.2f (%d itens)%n", total, itens.size()));
c.addObserver((itens, total) ->
    System.out.printf("Frete estimado: R$ %.2f%n", total > 200 ? 0.0 : 15.0));
```

Exemplo 3: Monitoramento de Métricas de Saúde do Sistema

Um sistema de monitoramento detecta quando CPU ou memória ultrapassam limiares. Diferentes observadores reagem: um loga o evento, outro envia alerta e outro executa escalonamento automático.

```
public interface MonitorObserver {
    void onMetricaAlterada(String metrica, double valor);
}

public class MonitorSistema {
    private final Map<String, Double> metricas = new HashMap<>();
    private final List<MonitorObserver> observers = new ArrayList<>();

    public void addObserver(MonitorObserver o) { observers.add(o); }

    public void atualizarMetrica(String nome, double valor) {
        metricas.put(nome, valor);
        observers.forEach(o -> o.onMetricaAlterada(nome, valor));
    }
}

// Observadores concretos:
MonitorSistema monitor = new MonitorSistema();

monitor.addObserver((m, v) -> {
    if ("CPU".equals(m) && v > 90)
        System.out.println("[LOG] CPU crítica: " + v + "%");
});
monitor.addObserver((m, v) -> {
    if ("MEM".equals(m) && v > 85)
        System.out.println("[ALERTA] Memória alta: " + v + "%");
});
monitor.addObserver((m, v) -> {
    if ("CPU".equals(m) && v > 95)
        System.out.println("[AUTO-SCALE] Provisionando nova instância...");
});

monitor.atualizarMetrica("CPU", 97.5);
```

3.7 State

 *Intenção: Permite que um objeto altere seu comportamento quando seu estado interno muda. O objeto parecerá ter mudado sua classe.*

Exemplo 1: Máquina de Venda Automática (Vending Machine)

Uma máquina de vendas possui estados: aguardando moeda, com crédito, dispensando produto e sem estoque. Cada estado define comportamentos diferentes para as mesmas operações.

```
public interface EstadoMaquina {
    void inserirMoeda(MaquinaVendas m);
    void selecionarProduto(MaquinaVendas m, String produto);
    void dispensar(MaquinaVendas m);
}
```

```

public class EstadoAguardando implements EstadoMaquina {
    public void inserirMoeda(MaquinaVendas m) {
        System.out.println("Moeda inserida. Selecione o produto.");
        m.setEstado(new EstadoComCredito());
    }
    public void selecionarProduto(MaquinaVendas m, String p) {
        System.out.println("Insira uma moeda primeiro.");
    }
    public void dispensar(MaquinaVendas m) {
        System.out.println("Sem crédito.");
    }
}

public class EstadoComCredito implements EstadoMaquina {
    private String produtoSelecionado;
    public void inserirMoeda(MaquinaVendas m) {
        System.out.println("Já há crédito. Selecione o produto.");
    }
    public void selecionarProduto(MaquinaVendas m, String p) {
        produtoSelecionado = p;
        System.out.println("Produto selecionado: " + p);
        m.setEstado(new EstadoDispensando(p));
    }
    public void dispensar(MaquinaVendas m) {
        System.out.println("Selecione um produto primeiro.");
    }
}

public class MaquinaVendas {
    private EstadoMaquina estadoAtual = new EstadoAguardando();
    public void setEstado(EstadoMaquina e) { estadoAtual = e; }
    public void inserirMoeda() { estadoAtual.inserirMoeda(this); }
    public void selecionar(String p) { estadoAtual.selecionarProduto(this, p); }
    public void dispensar() { estadoAtual.dispensar(this); }
}

```

Exemplo 2: Ciclo de Vida de Pedido no E-commerce

Um pedido percorre estados: PENDENTE → PAGO → EM_SEPARAÇÃO → ENVIADO → ENTREGUE. Cada transição tem regras de negócio específicas encapsuladas em cada estado.

```

public interface EstadoPedido {
    void pagar(Pedido p);
    void separar(Pedido p);
    void enviar(Pedido p);
    void entregar(Pedido p);
    void cancelar(Pedido p);
    String getDescricao();
}

public class EstadoPendente implements EstadoPedido {
    public void pagar(Pedido p) {
        System.out.println("Pagamento confirmado.");
        p.setEstado(new EstadoPago());
    }
    public void separar(Pedido p) { throw new IllegalStateException("Pedido não pago"); }
}

public void enviar(Pedido p) { throw new IllegalStateException("Pedido não pago"); }
}

public void entregar(Pedido p) { throw new IllegalStateException("Pedido não pago"); }
}

public void cancelar(Pedido p) {
    System.out.println("Pedido cancelado.");
    p.setEstado(new EstadoCancelado());
}
}

```

```

    public String getDescricao() { return "PENDENTE"; }
}

public class Pedido {
    private EstadoPedido estado = new EstadoPendente();
    private final String id;
    public Pedido(String id) { this.id = id; }
    public void setEstado(EstadoPedido e) { estado = e; }
    public void pagar() { estado.pagar(this); }
    public void separar() { estado.separar(this); }
    public void enviar() { estado.enviar(this); }
    public void entregar() { estado.entregar(this); }
    public void cancelar() { estado.cancelar(this); }
    public String getStatus() { return estado.getDescricao(); }
}

```

Exemplo 3: Controle de Semáforo em Simulação de Tráfego

Um semáforo muda de estado (vermelho → verde → amarelo → vermelho) ciclicamente. Cada estado tem duração e transição definidas, e o comportamento de carros reage ao estado atual.

```

public interface EstadoSemaforo {
    String getCor();
    int getDuracaoSegundos();
    EstadoSemaforo proximo();
    void agir(); // comportamento do tráfego
}

public class EstadoVermelho implements EstadoSemaforo {
    public String getCor() { return "VERMELHO"; }
    public int getDuracaoSegundos() { return 30; }
    public EstadoSemaforo proximo() { return new EstadoVerde(); }
    public void agir() { System.out.println("Carros parados."); }
}

public class EstadoVerde implements EstadoSemaforo {
    public String getCor() { return "VERDE"; }
    public int getDuracaoSegundos() { return 25; }
    public EstadoSemaforo proximo() { return new EstadoAmarelo(); }
    public void agir() { System.out.println("Carros passando."); }
}

public class EstadoAmarelo implements EstadoSemaforo {
    public String getCor() { return "AMARELO"; }
    public int getDuracaoSegundos() { return 5; }
    public EstadoSemaforo proximo() { return new EstadoVermelho(); }
    public void agir() { System.out.println("Carros desacelerando."); }
}

public class Semaforo {
    private EstadoSemaforo estado = new EstadoVermelho();
    public void ciclo() {
        estado.agir();
        System.out.printf("[%s] por %ds\n",
            estado.getCor(), estado.getDuracaoSegundos());
        estado = estado.proximo();
    }
}

```

3.8 Strategy

💡 Intenção: Define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis. Strategy permite que o algoritmo varie independentemente dos clientes que o utilizam.

Exemplo 1: Algoritmos de Ordenação Intercambiáveis

Um sistema de análise de dados precisa ordenar listas de diferentes tamanhos. A estratégia de ordenação é escolhida em tempo de execução: QuickSort para grandes volumes, InsertionSort para listas pequenas.

```
public interface EstrategiaOrdenacao<T extends Comparable<T>> {
    void ordenar(List<T> lista);
}

public class QuickSortStrategy<T extends Comparable<T>>
    implements EstrategiaOrdenacao<T> {
    public void ordenar(List<T> lista) {
        System.out.println("[QuickSort] Ordenando " + lista.size() + " elementos");
        Collections.sort(lista); // simplificado
    }
}

public class InsertionSortStrategy<T extends Comparable<T>>
    implements EstrategiaOrdenacao<T> {
    public void ordenar(List<T> lista) {
        System.out.println("[InsertionSort] Ordenando " + lista.size() + " elementos");
        for (int i = 1; i < lista.size(); i++) {
            T chave = lista.get(i); int j = i - 1;
            while (j >= 0 && lista.get(j).compareTo(chave) > 0) {
                lista.set(j + 1, lista.get(j)); j--;
            }
            lista.set(j + 1, chave);
        }
    }
}

public class ListaOrdenavel<T extends Comparable<T>> {
    private EstrategiaOrdenacao<T> estrategia;
    private final List<T> dados;

    public ListaOrdenavel(List<T> dados) { this.dados = dados; }
    public void setEstrategia(EstrategiaOrdenacao<T> e) { estrategia = e; }
    public void ordenar() {
        if (dados.size() > 100) estrategia = new QuickSortStrategy<>();
        else estrategia = new InsertionSortStrategy<>();
        estrategia.ordenar(dados);
    }
}
```

Exemplo 2: Cálculo de Frete por Transportadora

Uma loja pode calcular frete via Correios, FedEx ou retirada na loja. A estratégia de cálculo de frete é selecionada pelo cliente, sem alterar o código do pedido.

```
public interface EstrategiaFrete {
    double calcular(String cepOrigem, String cepDestino, double pesoKg);
    String getNomeTransportadora();
}

public class FreteCorreios implements EstrategiaFrete {
    public double calcular(String orig, String dest, double peso) {
        return 8.0 + (peso * 2.5); // PAC simplificado
    }
    public String getNomeTransportadora() { return "Correios (PAC)"; }
}
```

```

public class FreteFedEx implements EstrategiaFrete {
    public double calcular(String orig, String dest, double peso) {
        return 25.0 + (peso * 5.0); // Express
    }
    public String getNomeTransportadora() { return "FedEx Express"; }
}

public class FreteRetirada implements EstrategiaFrete {
    public double calcular(String o, String d, double p) { return 0.0; }
    public String getNomeTransportadora() { return "Retirada na Loja"; }
}

public class Pedido {
    private EstrategiaFrete frete;
    public void setFrete(EstrategiaFrete f) { this.frete = f; }
    public double calcularFrete(String cepOrig, String cepDest, double peso) {
        double val = frete.calcular(cepOrig, cepDest, peso);
        System.out.printf("%s: R$ %.2f%n", frete.getNomeTransportadora(), val);
        return val;
    }
}

```

Exemplo 3: Estratégias de Autenticação (OAuth, JWT, LDAP)

Um sistema corporativo suporta múltiplos métodos de autenticação conforme o tipo de usuário. A estratégia é definida dinamicamente, permitindo adicionar novos métodos sem modificar o sistema.

```

public interface EstrategiaAutenticacao {
    boolean autenticar(String credencial, String senha);
    String getTipo();
}

public class AutenticacaoJWT implements EstrategiaAutenticacao {
    public boolean autenticar(String token, String senha) {
        System.out.println("Validando JWT...");
        return token.startsWith("Bearer ");
    }
    public String getTipo() { return "JWT"; }
}

public class AutenticacaoLDAP implements EstrategiaAutenticacao {
    public boolean autenticar(String usuario, String senha) {
        System.out.println("Autenticando no LDAP: " + usuario);
        return !senha.isEmpty();
    }
    public String getTipo() { return "LDAP"; }
}

public class ServicoAutenticacao {
    private EstrategiaAutenticacao estrategia;
    public void setEstrategia(EstrategiaAutenticacao e) { estrategia = e; }
    public boolean login(String credencial, String senha) {
        System.out.println("Usando estratégia: " + estrategia.getTipo());
        return estrategia.autenticar(credencial, senha);
    }
}

// Uso com configuração dinâmica:
ServicoAutenticacao svc = new ServicoAutenticacao();
if (usuarioExterno) svc.setEstrategia(new AutenticacaoJWT());
else                 svc.setEstrategia(new AutenticacaoLDAP());
boolean ok = svc.login(credencial, senha);

```

3.9 Template Method

💡 Intenção: Define o esqueleto de um algoritmo em uma operação, adiando a definição de alguns passos para as subclasses. Permite que subclasses redefinam certos passos sem mudar a estrutura do algoritmo.

Exemplo 1: Geração de Relatórios em Diferentes Formatos

Todo relatório segue o mesmo fluxo: cabeçalho → dados → rodapé. O Template Method define esse fluxo na superclasse, e cada subclasses implementa o formato específico (HTML, CSV, JSON).

```
public abstract class GeradorRelatorio {

    // Template Method - define o fluxo
    public final String gerar(List<Map<String, Object>> dados) {
        StringBuilder sb = new StringBuilder();
        sb.append(gerarCabecalho());
        dados.forEach(linha -> sb.append(gerarLinha(linha)));
        sb.append(gerarRodape(dados.size()));
        return sb.toString();
    }

    protected abstract String gerarCabecalho();
    protected abstract String gerarLinha(Map<String, Object> linha);
    protected abstract String gerarRodape(int totalLinhas);
}

public class RelatorioHTML extends GeradorRelatorio {
    protected String gerarCabecalho() {
        return "<html><body><table><tr><th>Dados</th></tr>";
    }
    protected String gerarLinha(Map<String, Object> l) {
        return "<tr><td>" + l.values() + "</td></tr>";
    }
    protected String gerarRodape(int total) {
        return "</table><p>Total: " + total + "</p></body></html>";
    }
}

public class RelatorioCSV extends GeradorRelatorio {
    protected String gerarCabecalho() { return "ID;NOME;VALOR\n"; }
    protected String gerarLinha(Map<String, Object> l) {
        return String.join(";", l.values().stream()
            .map(Object::toString).toList()) + "\n";
    }
    protected String gerarRodape(int total) {
        return "# Total de registros: " + total + "\n";
    }
}
```

Exemplo 2: Pipeline de ETL (Extract, Transform, Load)

Processos de ETL seguem sempre: extrair dados da fonte → transformar/validar → carregar no destino. O Template Method define esse fluxo; subclasses especializam cada etapa.

```
public abstract class ETLPipeline<S, T> {

    // Template Method
    public final void executar() {
        System.out.println("Iniciando ETL...");
        List<S> dadosBrutos = extrair();
        System.out.println(dadosBrutos.size() + " registros extraídos");
        List<T> dadosTransformados = transformar(dadosBrutos);
    }
}
```

```

        System.out.println(dadosTransformados.size() + " registros transformados");
        carregar(dadosTransformados);
        System.out.println("ETL concluído.");
    }

    protected abstract List<S> extrair();
    protected abstract List<T> transformar(List<S> dados);
    protected abstract void carregar(List<T> dados);
}

public class ETLClientesCSV extends ETLPipeline<String, Cliente> {
    protected List<String> extrair() {
        System.out.println("Lendo clientes.csv...");
        return List.of("1;João;joao@email.com", "2;Maria;maria@email.com");
    }
    protected List<Cliente> transformar(List<String> linhas) {
        return linhas.stream().map(l -> {
            String[] p = l.split(";");
            return new Cliente(p[0], p[1], p[2]);
        }).toList();
    }
    protected void carregar(List<Cliente> clientes) {
        System.out.println("Inserindo " + clientes.size() + " no banco");
        // clienteRepository.saveAll(clientes);
    }
}

```

Exemplo 3: Processo de Validação de Cadastro

O processo de validação de cadastro segue: validar dados básicos → verificar duplicidade → aplicar regras de negócio → persistir. O Template Method define a sequência; subclasses especializam as regras.

```

public abstract class ProcessoCadastro<T> {

    // Template Method
    public final T processar(T entidade) {
        validarDadosBasicos(entidade);
        verificarDuplicidade(entidade);
        aplicarRegrasDeNegocio(entidade);
        T salvo = persistir(entidade);
        notificarSucesso(salvo); // hook - pode ser sobrescrito
        return salvo;
    }

    protected abstract void validarDadosBasicos(T e);
    protected abstract void verificarDuplicidade(T e);
    protected abstract void aplicarRegrasDeNegocio(T e);
    protected abstract T persistir(T e);

    // Hook com implementação padrão
    protected void notificarSucesso(T e) {
        System.out.println("Cadastro concluído: " + e);
    }
}

public class CadastroAluno extends ProcessoCadastro<Aluno> {
    protected void validarDadosBasicos(Aluno a) {
        if (a.getNome() == null || a.getNome().isBlank())
            throw new IllegalArgumentException("Nome obrigatório");
    }
    protected void verificarDuplicidade(Aluno a) {
        System.out.println("Verificando RA duplicado...");
    }
    protected void aplicarRegrasDeNegocio(Aluno a) {

```

```

        a.setRA(gerarRA()); // gera RA automático
    }
    protected Aluno persistir(Aluno a) {
        System.out.println("Salvando aluno: " + a.getNome()); return a;
    }
    private String gerarRA() { return "PUC-" + System.currentTimeMillis(); }
}

```

3.10 Visitor

💡 Intenção: Representa uma operação a ser realizada nos elementos de uma estrutura de objetos. O Visitor permite definir uma nova operação sem mudar as classes dos elementos sobre os quais opera.

Exemplo 1: Cálculo de Imposto por Tipo de Produto

Um sistema fiscal calcula impostos diferentes para produtos eletrônicos, alimentos e vestuário. O Visitor separa o algoritmo de cálculo dos objetos produto, facilitando a adição de novas tributações.

```

public interface ProdutoVisitable {
    double aceitar(VisitorImposto visitor);
}

public class ProdutoEletronico implements ProdutoVisitable {
    private final double preco;
    public ProdutoEletronico(double p) { preco = p; }
    public double getPreco() { return preco; }
    public double aceitar(VisitorImposto v) { return v.visitar(this); }
}

public class ProdutoAlimento implements ProdutoVisitable {
    private final double preco; private final boolean basico;
    public ProdutoAlimento(double p, boolean basico) {
        preco = p; this.basico = basico;
    }
    public double getPreco() { return preco; }
    public boolean isBasico() { return basico; }
    public double aceitar(VisitorImposto v) { return v.visitar(this); }
}

public interface VisitorImposto {
    double visitar(ProdutoEletronico p);
    double visitar(ProdutoAlimento p);
}

public class CalculadoraICMS implements VisitorImposto {
    public double visitar(ProdutoEletronico p) {
        return p.getPreco() * 0.25; // 25% ICMS eletrônicos
    }
    public double visitar(ProdutoAlimento p) {
        return p.isBasico() ? 0.0 : p.getPreco() * 0.12;
    }
}

// Uso:
List<ProdutoVisitable> produtos = List.of(
    new ProdutoEletronico(2000.0),
    new ProdutoAlimento(5.0, true));
VisitorImposto calc = new CalculadoraICMS();
double totalImposto = produtos.stream()
    .mapToDouble(p -> p.aceitar(calc)).sum();

```

Exemplo 2: Serialização de Árvore de Expressão Matemática

Uma calculadora armazena expressões como árvore (soma, multiplicação, número). O Visitor percorre a árvore e gera diferentes representações (LaTeX, string infixada, avaliação numérica).

```
public interface Expressao {
    <T> T aceitar(ExpressaoVisitor<T> v);
}

public class Numero implements Expressao {
    public final double valor;
    public Numero(double v) { valor = v; }
    public <T> T aceitar(ExpressaoVisitor<T> v) { return v.visitar(this); }
}

public class Soma implements Expressao {
    public final Expressao esq, dir;
    public Soma(Expressao e, Expressao d) { esq = e; dir = d; }
    public <T> T aceitar(ExpressaoVisitor<T> v) { return v.visitar(this); }
}

public interface ExpressaoVisitor<T> {
    T visitar(Numero n);
    T visitar(Soma s);
}

public class Avaliador implements ExpressaoVisitor<Double> {
    public Double visitar(Numero n) { return n.valor; }
    public Double visitar(Soma s) {
        return s.esq.aceitar(this) + s.dir.aceitar(this);
    }
}

public class LaTeXPrinter implements ExpressaoVisitor<String> {
    public String visitar(Numero n) { return String.valueOf(n.valor); }
    public String visitar(Soma s) {
        return s.esq.aceitar(this) + " + " + s.dir.aceitar(this);
    }
}

// Uso: (3 + 4) + 2
Expressao expr = new Soma(new Soma(new Numero(3), new Numero(4)), new Numero(2));
System.out.println(expr.aceitar(new Avaliador())); // 9.0
System.out.println(expr.aceitar(new LaTeXPrinter())); // 3.0 + 4.0 + 2.0
```

Exemplo 3: Análise Estática de Código (Lint)

Uma ferramenta de análise percorre a AST de um programa Java. Cada tipo de nó (método, variável, chamada) é visitado por diferentes verificadores (nomes de variáveis, complexidade ciclomática).

```
public interface NoAST {
    void aceitar(AnalizadorVisitor v);
}

public class DeclaracaoMetodo implements NoAST {
    public final String nome; public final int numLinhas;
    public DeclaracaoMetodo(String n, int l) { nome = n; numLinhas = l; }
    public void aceitar(AnalizadorVisitor v) { v.visitar(this); }
}

public class DeclaracaoVariavel implements NoAST {
    public final String nome; public final String tipo;
    public DeclaracaoVariavel(String n, String t) { nome = n; tipo = t; }
    public void aceitar(AnalizadorVisitor v) { v.visitar(this); }
}
```

```

public interface AnalisadorVisitor {
    void visitar(DeclaracaoMetodo m);
    void visitar(DeclaracaoVariavel v);
}

public class VerificadorConvencaoNomes implements AnalisadorVisitor {
    public void visitar(DeclaracaoMetodo m) {
        if (Character.isUpperCase(m.nome.charAt(0)))
            System.out.println("WARN: Método '" + m.nome + "' deve começar com
minúscula");
    }
    public void visitar(DeclaracaoVariavel v) {
        if (v.nome.contains("_"))
            System.out.println("WARN: Variável '" + v.nome + "' com underscore");
    }
}

public class VerificadorTamanhoMetodo implements AnalisadorVisitor {
    public void visitar(DeclaracaoMetodo m) {
        if (m.numLinhas > 20)
            System.out.println("WARN: Método '" + m.nome + "' muito grande: " +
m.numLinhas);
    }
    public void visitar(DeclaracaoVariavel v) { /* sem verificação */ }
}

```

3.11 Interpreter

💡 Intenção: Dada uma linguagem, define uma representação para sua gramática juntamente com um interpretador que usa a representação para interpretar sentenças na linguagem.

Exemplo 1: Interpretador de Expressões Booleanas

Um sistema de regras de negócio avalia condições como 'usuarioPremium AND (saldoPositivo OR cupomValido)'. O Interpreter representa e avalia essas expressões booleanas compostas.

```

public interface ExpressaoBooleana {
    boolean interpretar(Map<String, Boolean> contexto);
}

public class VariavelBooleana implements ExpressaoBooleana {
    private final String nome;
    public VariavelBooleana(String n) { nome = n; }
    public boolean interpretar(Map<String, Boolean> ctx) {
        return Boolean.TRUE.equals(ctx.get(nome));
    }
}

public class AndExpressao implements ExpressaoBooleana {
    private final ExpressaoBooleana esq, dir;
    public AndExpressao(ExpressaoBooleana e, ExpressaoBooleana d) {
        esq = e; dir = d;
    }
    public boolean interpretar(Map<String, Boolean> ctx) {
        return esq.interpretar(ctx) && dir.interpretar(ctx);
    }
}

public class OrExpressao implements ExpressaoBooleana {
    private final ExpressaoBooleana esq, dir;
    public OrExpressao(ExpressaoBooleana e, ExpressaoBooleana d) {
        esq = e; dir = d;
    }
}

```

```

    }
    public boolean interpretar(Map<String, Boolean> ctx) {
        return esq.interpretar(ctx) || dir.interpretar(ctx);
    }
}

// Uso: usuarioPremium AND (saldoPositivo OR cupomValido)
ExpressaoBooleana regra = new AndExpressao(
    new VariavelBooleana("usuarioPremium"),
    new OrExpressao(
        new VariavelBooleana("saldoPositivo"),
        new VariavelBooleana("cupomValido")));

Map<String, Boolean> ctx = Map.of(
    "usuarioPremium", true,
    "saldoPositivo", false,
    "cupomValido", true);

System.out.println(regra.interpretar(ctx)); // true

```

Exemplo 2: Mini Linguagem de Consulta (Filtro de Dados)

Uma ferramenta de BI permite que usuários escrevam filtros como 'preço > 100 AND categoria = "eletronicos"'. O Interpretar analisa e avalia essas expressões sobre coleções de produtos.

```

public interface Predicado<T> {
    boolean avaliar(T objeto);
}

public class PredicadoIgual implements Predicado<Map<String, Object>> {
    private final String campo; private final Object valor;
    public PredicadoIgual(String c, Object v) { campo = c; valor = v; }
    public boolean avaliar(Map<String, Object> o) {
        return valor.equals(o.get(campo));
    }
}

public class PredicadoMaiorQue implements Predicado<Map<String, Object>> {
    private final String campo; private final double limite;
    public PredicadoMaiorQue(String c, double l) { campo = c; limite = l; }
    public boolean avaliar(Map<String, Object> o) {
        return ((Number) o.get(campo)).doubleValue() > limite;
    }
}

public class PredicadoAnd<T> implements Predicado<T> {
    private final Predicado<T> esq, dir;
    public PredicadoAnd(Predicado<T> e, Predicado<T> d) { esq=e; dir=d; }
    public boolean avaliar(T o) { return esq.avaliar(o) && dir.avaliar(o); }
}

// Uso: preço > 100 AND categoria = "eletronicos"
Predicado<Map<String, Object>> filtro = new PredicadoAnd<>(
    new PredicadoMaiorQue("preço", 100.0),
    new PredicadoIgual("categoria", "eletronicos"));

List<Map<String, Object>> resultados = produtos.stream()
    .filter(filtro::avaliar).toList();

```

Exemplo 3: Interpretador de Comandos de Terminal

Um sistema embarcado aceita comandos textuais simples ('LIGAR MOTOR 1', 'DESLIGAR BOMBA 2'). O Interpretar converte esses comandos em ações concretas do sistema.

```

public interface ComandoTerminal {
    void executar(SistemaEmbarcado sistema);
}

```

```
}

public class ComandoLigar implements ComandoTerminal {
    private final String dispositivo; private final int id;
    public ComandoLigar(String d, int i) { dispositivo = d; id = i; }
    public void executar(SistemaEmbarcado s) {
        s.ligar(dispositivo, id);
    }
}

public class ComandoDesligar implements ComandoTerminal {
    private final String dispositivo; private final int id;
    public ComandoDesligar(String d, int i) { dispositivo = d; id = i; }
    public void executar(SistemaEmbarcado s) {
        s.desligar(dispositivo, id);
    }
}

public class InterpretadorComandos {
    public ComandoTerminal interpretar(String linha) {
        String[] partes = linha.toUpperCase().split(" ");
        String acao = partes[0];
        String disp = partes[1];
        int id      = Integer.parseInt(partes[2]);

        return switch (acao) {
            case "LIGAR"      -> new ComandoLigar(disp, id);
            case "DESLIGAR" -> new ComandoDesligar(disp, id);
            default -> throw new IllegalArgumentException("Comando inválido: " + acao);
        };
    }
}

// Uso:
InterpretadorComandos interp = new InterpretadorComandos();
interp.interpretar("LIGAR MOTOR 1").executar(sistema);
interp.interpretar("DESLIGAR BOMBA 2").executar(sistema);
```

Referências e Leituras Recomendadas

- GAMMA, Erich et al. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
 - FREEMAN, Eric; ROBSON, Elisabeth. Head First Design Patterns. 2. ed. O'Reilly Media, 2021.
 - MARTIN, Robert C. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008.
 - BLOCH, Joshua. Effective Java. 3. ed. Addison-Wesley, 2018.
 - SHVETS, Alexander. Dive Into Design Patterns. Refactoring Guru, 2021. Disponível em: <https://refactoring.guru/design-patterns>
 - ORACLE. Java Documentation — JDK 17. Disponível em: <https://docs.oracle.com/en/java/javase/17/>
-